CSE4019 – Advanced Java Programming

Course Type: LP

Credits: 3

Assessment Weightage

Sl.No.	Mark Title	Max. Mark	Weightage %
1	Mid Term	50	30
2	Lab Assessment - Continuous Assessment	20	20
3	Lab Assessment - Challenging Task	15	15
4	Attendance	5	5
5	Term End Examination	100	30

Assessment Weightage

S.No	Assessment Title	Assessment Followed	Marks	Due Date
1	Lab Continuous Evaluation (20 Marks)	Digital Lab Report	10	3rd April 2026
		Assignment 1	5	4th March 2026
		Assignment 2	5	6th April 2026
2	Challenging Task (15 Marks)	Project (Group)	10	6th April 2026
		Quiz/ Class Activity submissions	5	8th April 2026

Note: Follow the due dates without fail

Failed to submit any submission on or before due date will be awarded as ZERO marks

Syllabus

Unit No	Module Description	No of Hours	SO
1	**Object Oriented Programming using Java:** Overview of the Java platform, Introduction to Java: Java Architecture, Advantages of Java, PATH and CLASSPATH variables, Compiling and Running Java Programs, Class and Object: Creating Classes, Interfaces, Creating Objects, Access Modifiers, Arrays, Packages, Inheritance, Exception Handling and Threading: Try, Catch, Finally, Throws, Creating Multithreaded Programs, Thread Life Cycle, Input/Output Serialization and Object Serialization.	6	a, b, c
2	**Java Beans:** Common Elements of Graphical User Interfaces: Introduction, Main features and Technology of GUI, Introducing the Java foundation classes, Event Model, JFC Sample programs, Layout managers, Events, Java Beans : Introduction, JavaBeans Component Model, Bean Development Environments, Using the Sun BeanBox, Creating a JavaBean Class, Exploring JavaBean Property Types, Adding Custom Event Types, Creating a JavaBean Class with Events, Using the BeanInfo Classes.	6	a, b, c
3	**JDBC** Using Relational Databases: Introduction, Best Practices for Programming for Databases, JDBC Drivers for RDBM Systems, SQL to Java Type Mapping, Using the java.sql API, Coding Transactions, Using the Javax.sql API, connection pooling .	6	b, c

Syllabus

	.		
4	**SWING** Swing and MVC Design Patterns: Design Pattern, MVC Pattern, MVC Analysis of Swing Buttons. Layout Management: Border Layout, Grid Layout, Gridbag Layout, Group Layout, Using No Layout managers, Custom layout Managers Text Input: Text Fields, Password Fields, Text Areas, Scroll Pane, Labeling Components. Choice Components: Check Boxes, Radio Buttons, Borders, Combo Boxes, Sliders. Menus: Menu Building, Icons in Menu Items, Check box and Radio Buttons in Menu Items, Pop-up Menus, Keyboard Mnemonics and Accelerators, Enabling and Design menu Items, Toolbars, Tooltips. Dialog Boxes: Option Dialogs, Creating Dialogs, Data Exchange, File Choosers, Color Choosers .Components Organizers: Split Panes, Tabbed Panes, Desktop Panes and Internal Frames, Cascading and Tiling Advance Swing Components: List, Trees, Tables, Progress Bars.	6	b, c
5	**Applets & EJB:** Applet Basics, Applet HTML tags & Attribute, Multimedia, URL Encapsulation, JAR files, Application Packaging, Storage of Application Preferences. Networking Programming Introduction, Working with URLs, Working with Sockets, Remote Method Invocation. Building Web Application Introduction, The Techology of the Web,Servlets,The Servlet API Introduction, Enterprising Programming,What are EJBs, session EJBs, EJB Clients, Entity EJBs,Message-Driven Beans,EJB Transaction Characterstics,EJB security.	6	a, c, k

Course Objectives

- To understand how to write, compile and execute a simple Java Program.
- Understand fundamentals of object-oriented programming in Java, including defining classes, invoking methods, creating multithreaded programs, etc.
- Be aware of the important topics and principles of Java foundation classes.
- Have the ability to write a computer program to solve specified problems.
- Be able to create web page using Applets
- Create, debug and run simple Java programs.

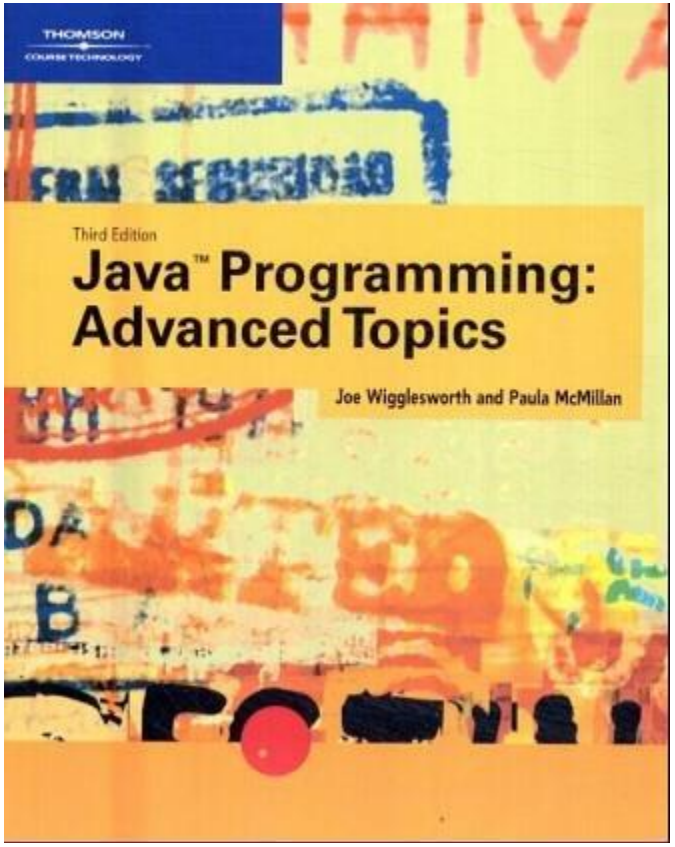
Course Outcomes

Students who complete this course will be able:

- To understand the basic principles of Java Programming.
- To Embed Java Applet into a web page.
- To understand the difference of Java beans and EJB.
- To design graphical user interface using swing.
- To make JDBC connection using queries.

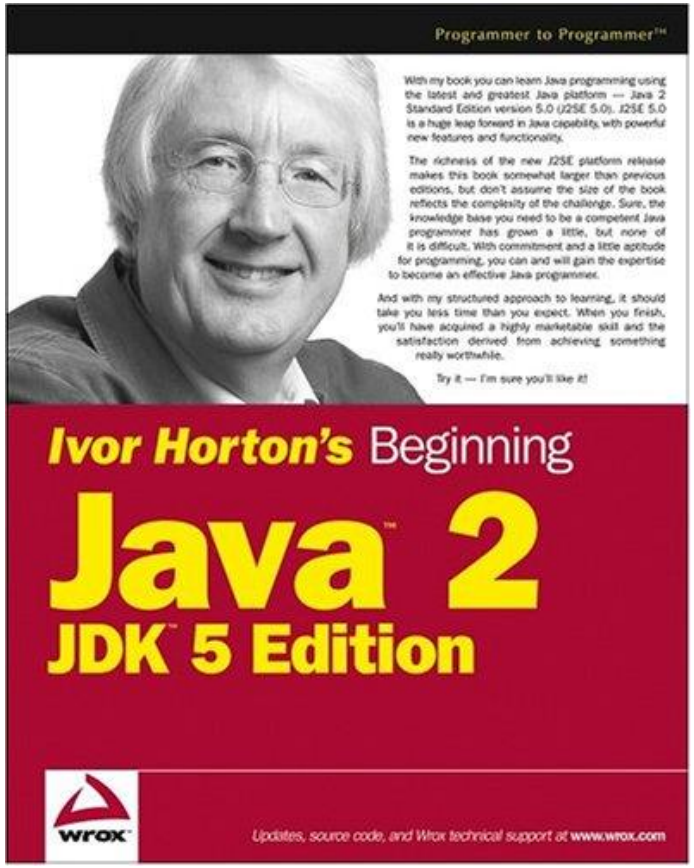
Text Book (Prescribed)

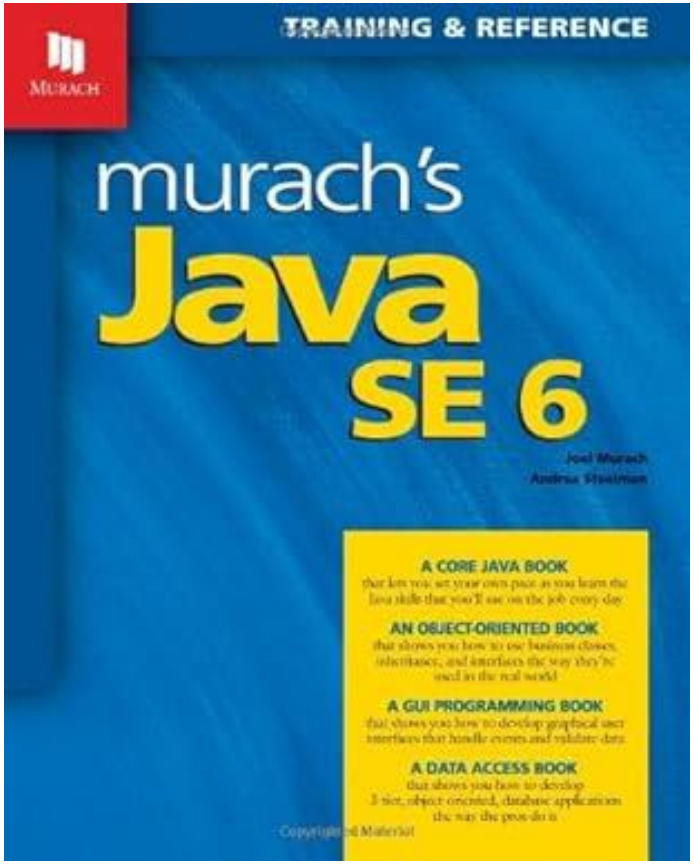
Joe Wiggles Worth and Paula Mc Millan, "**Java programming: Advanced Topics**", Third Edition,Thomson.



Text Book (References)

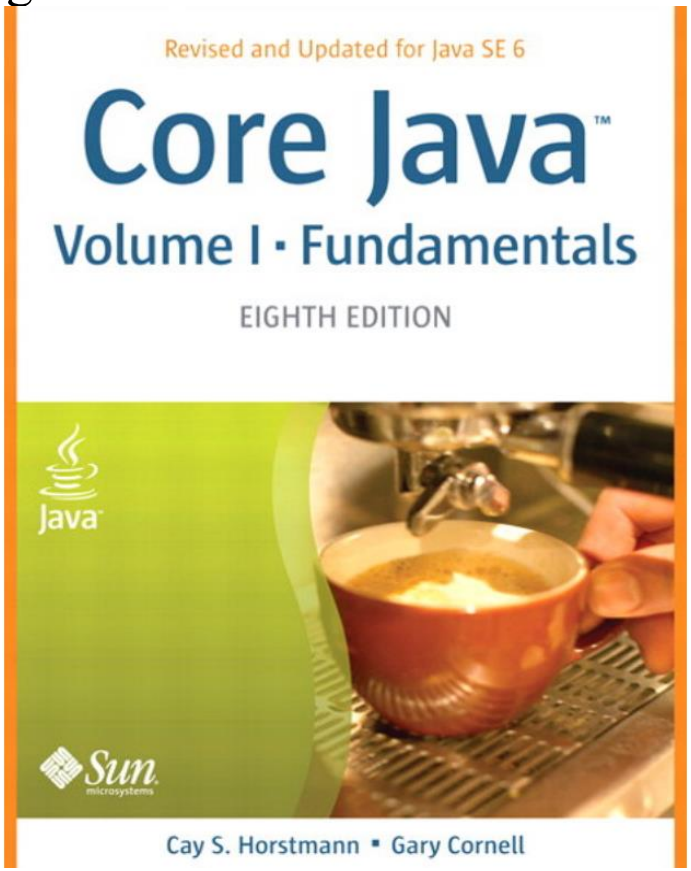
1. Ivor Horton's, "Beginning Java 2- JDK 5 Edition", Wrox (2008)
2. Joel Murach,Andrea Steelman "Java SE 6" , SPD

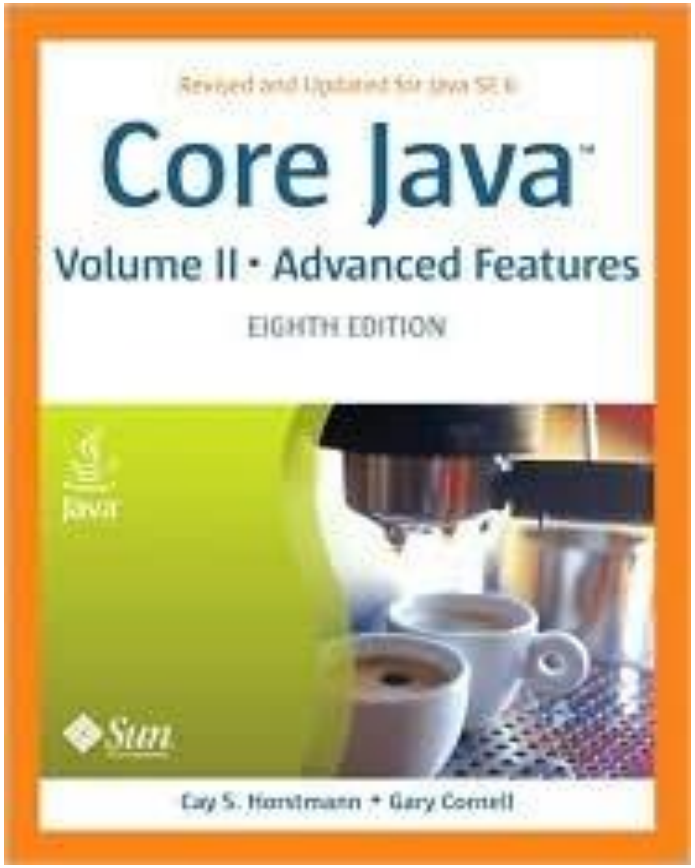




Text Book (References)

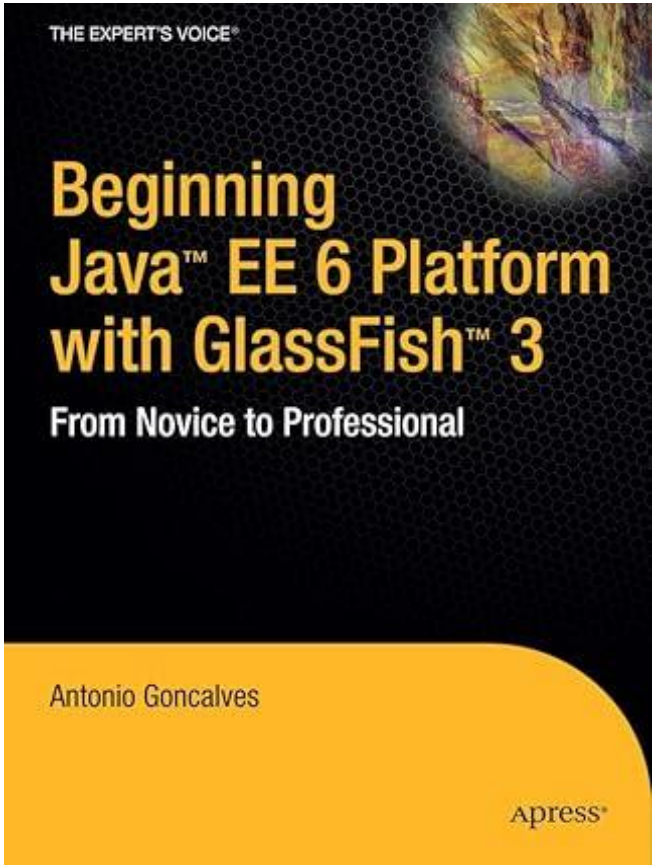
1. Cay Horstmann and Grazy Cornell, Core Java Volume I-Fundamentals, Eighth Edition
2. Cay Horstmann and Grazy Cornell, Core Java Volume II-Advance Features, Eighth Edition





Text Book (References)

1. "Advanced Java 2 Platform HOW TO PROGRAM" by H. M.Deitel, P. J. Deitel, S. E. Santry – Prentice Hall
2. "Beginning Java™ EE 6 Platform with GlassFish 3 From Novice to Professional" by Antonio Goncalves.



List of Experiments

Indicative List of Experiments:	SO-k
1. Java program to print 'Hello world' 1.1 How to accept input from keyboard in java? • Using Scanner Class • Using Console Class • Using InputStreamReader and BufferedReader Class 2. Implement Arrays, Packages, Inheritance, Exception Handling, Serialization and Threading in Java. 3. Java program to find area and perimeter of a circle using class 4. Java program to print a rectangle using stars. 5. Create and Execute Java Bean class. 6. Implementation of JDBC connection with MySql/Oracle.	
7. Drawing Shapes (circle, waveform, enable and disable buttons) using Swing. 8. Perform connection-oriented Socket Programming in networking. 9. Java Applet program to show the animation of a bouncing ball 10. Creating Web application using Servlet.	

Advanced Java Programming

Module 1

Advanced Java Programming Module 1

1	**Object Oriented Programming using Java:** Overview of the Java platform, Introduction to Java: Java Architecture, Advantages of Java, PATH and CLASSPATH variables, Compiling and Running Java Programs, Class and Object: Creating Classes, Interfaces, Creating Objects, Access Modifiers, Arrays, Packages, Inheritance, Exception Handling and Threading: Try, Catch, Finally, Throws, Creating Multithreaded Programs, Thread Life Cycle, Input/Output Serialization and Object Serialization.

Java

- Java is an object-oriented programming language that runs on almost all electronic devices and is the most common language used by web developers.
- Java is platform-independent because of Java virtual machines (JVMs).
- It follows the principle of "write once, run everywhere.”
- When a JVM is installed on the host operating system, it automatically adapts to the environment and executes the program’s functionalities.
- To install Java on a computer, the developer must download the JDK and set up the Java Runtime Environment (JRE).
- Java offers many benefits, which we will explore in detail.

Features / Advantages of Java

- There is given many features of java. They are also known as java buzzwords. The Java Features given below are simple and easy to understand.
  - Simple
  - Object-Oriented
  - Platform independent
  - Secured
  - Robust
  - Architecture neutral
  - Portable
  - Dynamic
  - Interpreted
  - High Performance
  - Multithreaded
  - Distributed

Simple

According to Sun, Java language is simple because:

- syntax is based on C++ (so easier for programmers to learn it after C++).
- removed many confusing and/or rarely-used features e.g., explicit pointers, operator overloading etc.
- No need to remove unreferenced objects because there is Automatic Garbage Collection in java.

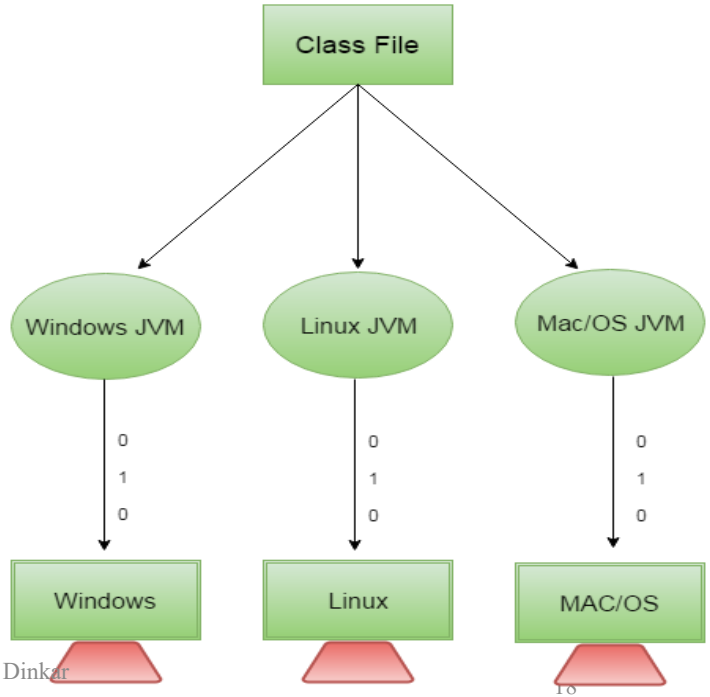
Object-oriented

- Object-oriented means we organize our software as a combination of different types of objects that incorporates both data and behaviour.
- Object-oriented programming(OOPs) is a methodology that simplify software development and maintenance by providing some rules.
- Basic concepts of OOPs are:
  - **Object**
  - **Class**
  - **Inheritance**
  - **Polymorphism**
  - **Abstraction**
  - **Encapsulation**

Platform Independent

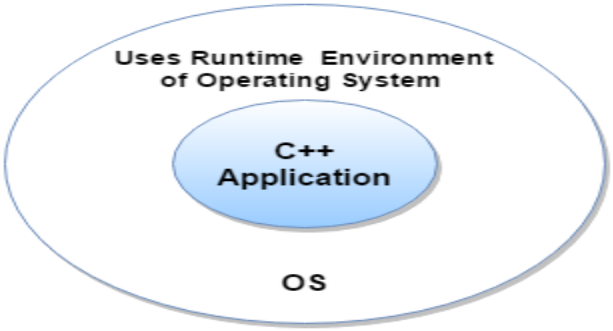
- A platform is the hardware or software environment in which a program runs.
- There are two types of platforms software-based and hardware-based. Java provides software-based platform.
- The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on the top of other hardware-based platforms. It has two components:
- **Runtime Environment**
- **API(Application Programming Interface)**

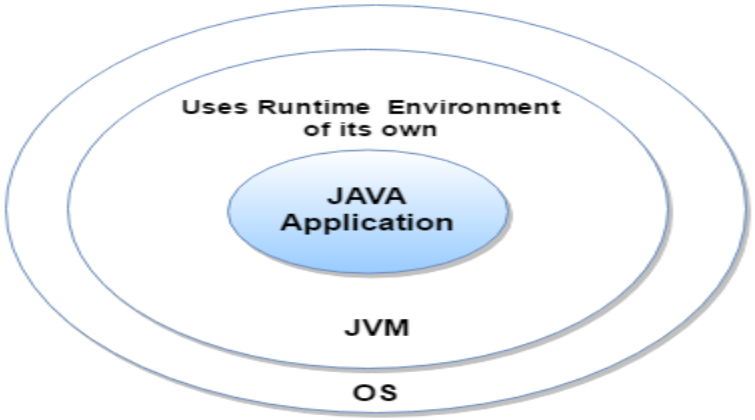
- Java code can be run on multiple platforms e.g. Windows, Linux, Sun Solaris, Mac/OS etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms i.e. Write Once and Run Anywhere(WORA).



Secured

- Java is secured because:
- **No explicit pointer**
- **Java Programs run inside virtual machine sandbox**
- **Classloader:** adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier:** checks the code fragments for illegal code that can violate access right to objects.
- **Security Manager:** determines what resources a class can access such as reading and writing to the local disk.
- These security are provided by java language. Some security can also be provided by application developer through SSL, JAAS, Cryptography etc.





Robust

- Robust simply means strong. Java uses strong memory management.
- There are lack of pointers that avoids security problem.
- There is automatic garbage collection in java. There is exception handling and type checking mechanism in java. All these points makes java robust.

Architecture-neutral

- There is no implementation dependent features e.g. size of primitive types is fixed.
- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. But in java, it occupies 4 bytes of memory for both 32 and 64 bit architectures.

Portable

- We may carry the java bytecode to any platform.

High-performance

- Java is faster than traditional interpretation since byte code is "close" to native code still somewhat slower than a compiled language (e.g., C++)

Distributed

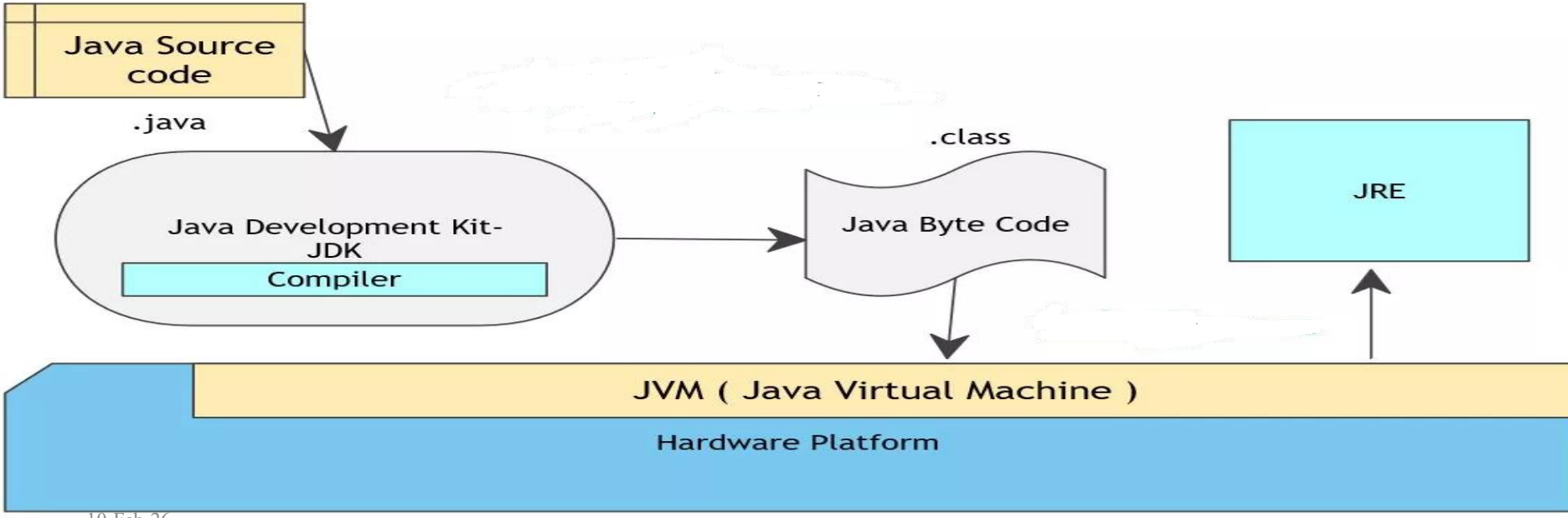
- We can create distributed applications in java.
- Remote Method Invocation (RMI) and Enterprise JavaBeans (EJB) are used for creating distributed applications.
- We may access files by calling the methods from any machine on the internet.

Multi-threaded

- A thread is like a separate program, executing concurrently.
- We can write Java programs that deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it doesn't occupy memory for each thread.
- It shares a common memory area.
- Threads are important for multi-media, Web applications etc.

Overview of Java Platform

- A platform is an environment in which Java applications run/execute.
- There are two types of platforms hardware-based and software-based.
- Java Platform provides a software-based platform for running Java applications.
- The Java platform differs from other platforms in the sense that it's a software-based platform that runs on top of hardware-based platforms.
- Java Platform has many components like:



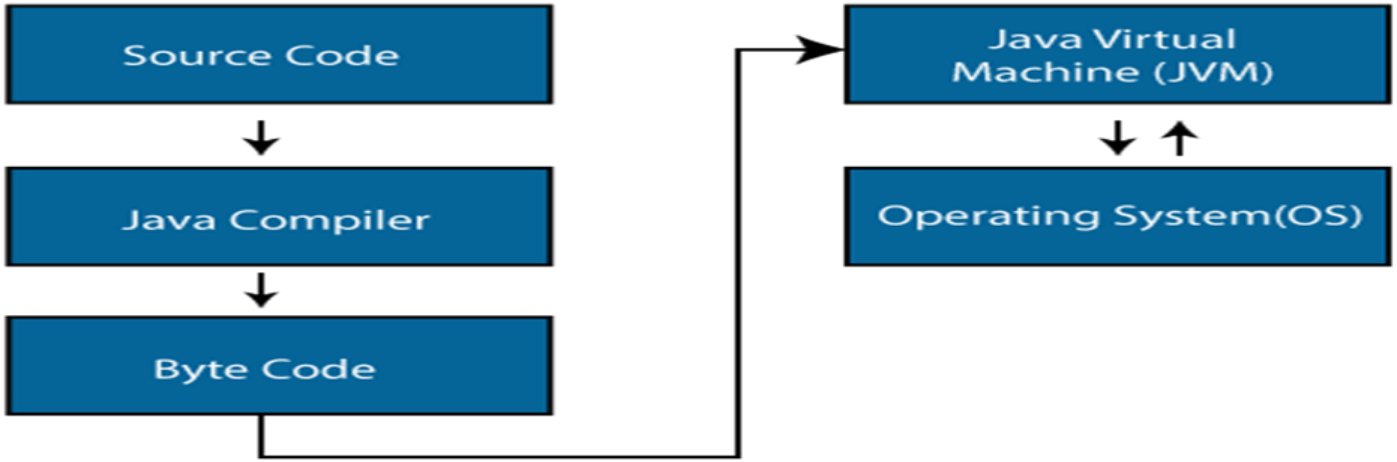
Overview of Java Platform

- Java Platform has many components like:
  - Java Virtual Machine
  - Runtime Environment
  - Java API (Application Programming Interface with JDK)
  - Java Development Tools, etc

- The different Java editions are as follows:
  - Java Standard Edition (Java SE)
  - Java Enterprise Edition (Java EE)
  - Java Micro Edition (Java ME)

Java Architecture

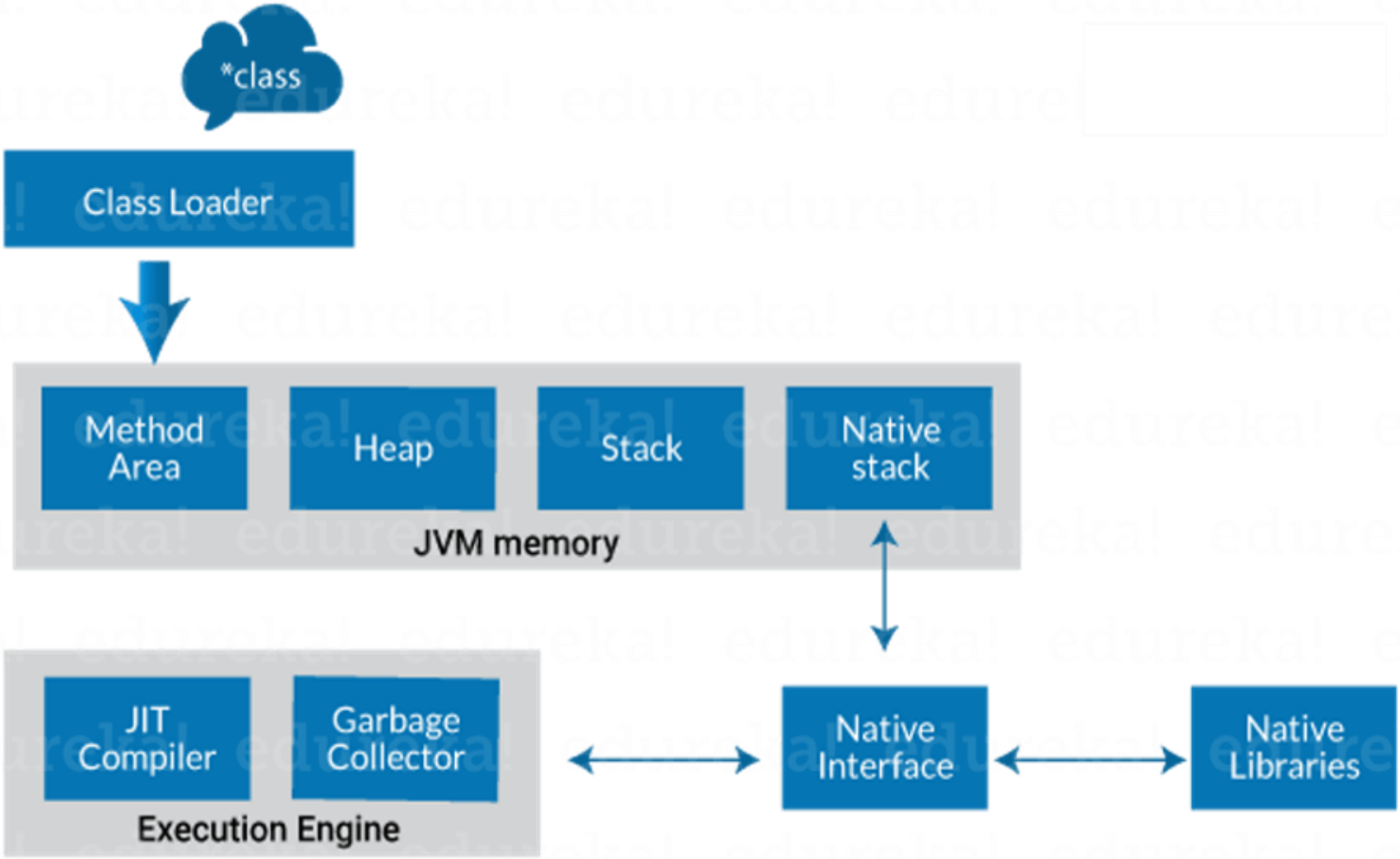
- Java Architecture combines the process of compilation and interpretation.
- In Java, there is a process of compilation and interpretation.
  - The code written in Java, is converted into byte codes which is done by the Java Compiler.
  - The byte codes, then are converted into machine code by the JVM.
  - The Machine code is executed directly by the machine.



Java Architecture

- **Components of Java Architecture:**
  - There are three main components of Java language: ***JVM, JRE, and JDK***.
- **Java Virtual Machine:**
  - Write once Run Anywhere : because of their ability to run a code on any platform.
  - This is done only because of JVM - is a Java platform component that provides an environment for executing Java programs.
  - JVM interprets the bytecode into machine code which is executed in the machine in which the Java program runs.
  - JVM performs:
    - Loads the code
    - Verifies the code
    - Executes the code
    - Provides runtime environment

A **Java virtual machine** (**JVM**) is an abstract computing **machine** that enables a computer to run a **Java** program. There are three notions of the **JVM** specification, implementation and instance.



Java Architecture

- **Class Loader**: Class loader is a subsystem of JVM. It is used to load class files.
  - Whenever we run the java program, class loader loads it first.
- **Class method area**: It is one of the Data Area in JVM, in which Class data will be stored. Static Variables, Static Blocks, Static Methods, Instance Methods are stored in this area.
- **Heap**: A heap is created when the JVM starts up. It may increase or decrease in size while the application runs.
- **Stack**: JVM stack is known as a thread stack. It is a data area in the JVM memory which is created for a single execution thread. The JVM stack of a thread is used by the thread to store various elements i.e.; local variables, partial results, and data for calling method and returns.
- **Native stack**: It subsumes all the native methods used in your application.
- **Execution Engine:**
  - JIT compiler : Just in time compiler - compiling bytecodes to machine code at run time – enabled default
  - Garbage collector

Compiling and Running Java Programs

- Java, being a platform-independent programming language, doesn't work on the one-step compilation.
- Instead, it involves a two-step execution, first through an OS-independent compiler; and second, in a virtual machine (JVM) which is custom-built for every operating system.
- The two principal stages are explained below:
  - **Compiler:**
    - First, the source '.java' file is passed through the compiler, which then encodes the source code into a machine-independent encoding, known as Bytecode.

Compiling and Running Java Programs

- Compiler:
  - The content of each class contained in the source file is stored in a separate '.class' file.
    - While converting the source code into the bytecode, the compiler follows the following steps:
      - Step 1: **Parse**: Reads a set of *.java source files and maps the resulting token sequence into AST (Abstract Syntax Tree)-Nodes.
      - Step 2: **Enter**: Enters symbols for the definitions into the symbol table.
      - Step 3: **Process annotations**: If Requested, processes annotations found in the specified compilation units.
      - Step 4: **Attribut**e: Attributes the Syntax trees. This step includes name resolution, type checking and constant folding.
      - Step 5: **Flow**: Performs dataflow analysis on the trees from the previous step. This includes checks for assignments and reachability.

Compiling and Running Java Programs

- Compiler:
  - Step 6: **Desugar**: Rewrites the AST and translates away some syntactic sugar.
  - Step 7: **Generate**: Generates '.Class' files.

- Execution:
  - The class files generated by the compiler are independent of the machine or the OS, which allows them to be run on any system.
  - To run, the main class file (the class that contains the method main) is passed to the JVM and then goes through three main stages before the final machine code is executed. These stages are:

    1. Class Loader
    2. Bytecode Verifier
    3. Just-In-Time Compiler

Compiling and Running Java Programs

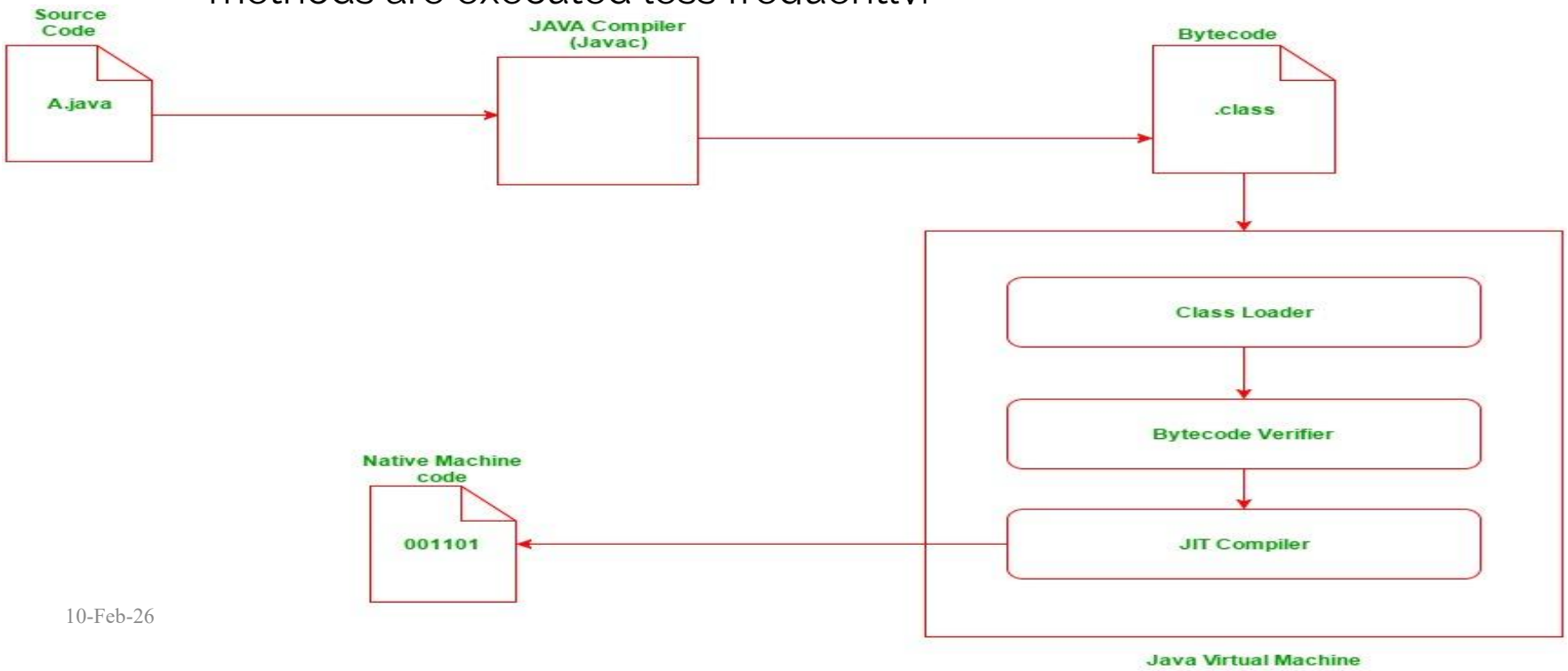
- Execution:
  - Class Loader
    - Class loaders are responsible for **loading Java classes dynamically to the JVM (Java Virtual Machine) during runtime.**
    - They're also part of the JRE (Java Runtime Environment). Therefore, the JVM doesn't need to know about the underlying files or file systems in order to run Java programs thanks to class loaders.
    - Furthermore, these Java classes aren't loaded into memory all at once, but rather when they're required by an application.
    - This is where class loaders come into the picture. They're responsible for loading classes into memory.
    - There are two types of class loaders : **primordial and non-primordial**
      - The primordial class loader is embedded into all the JVMs and is the default class loader.
      - A non-primordial class loader is a user-defined class loader, which can be coded in order to customize the class-loading process. Non-primordial class loader, if defined, is preferred over the default one, to load

Compiling and Running Java Programs

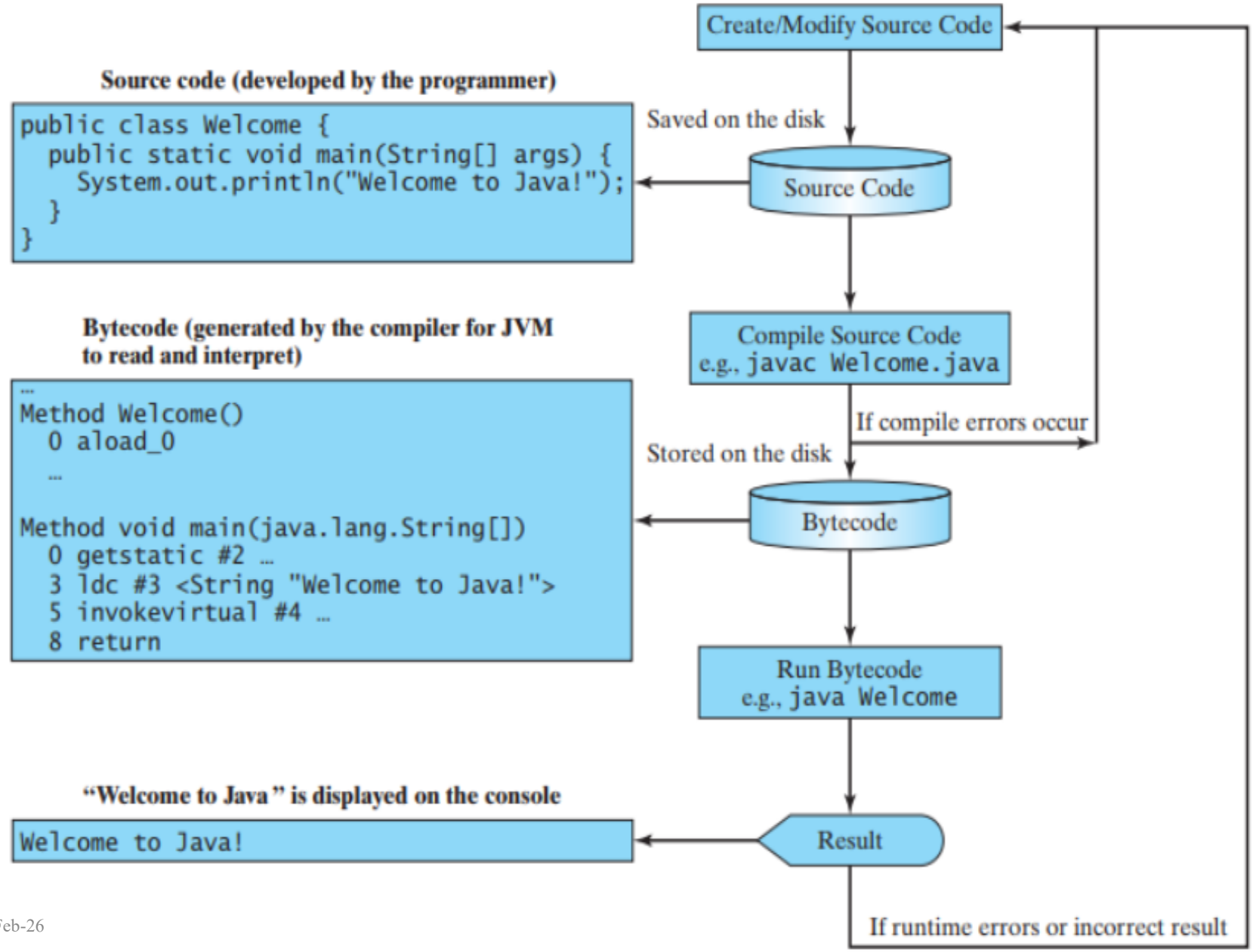
- Execution:
  - Bytecode Verifier
    - After the bytecode of a class is loaded by the class loader, it has to be inspected by the bytecode verifier, whose job is to check that the instructions don't perform damaging actions.
    - The following are some of the checks carried out:
      - Variables are initialized before they are used.
      - Method calls match the types of object references.
      - Rules for accessing private data and methods are not violated.
      - Local variable accesses fall within the runtime stack.
      - The run-time stack does not overflow.
      - If any of the above checks fail, the verifier doesn't allow the class to be loaded.
  - Just-In-Time Compiler
    - This is the final stage encountered by the java program, and its job is to convert the loaded bytecode into machine code.

Compiling and Running Java Programs

- Execution:
  - Just-In-Time Compiler
    - This is the final stage encountered by the java program, and its job is to convert the loaded bytecode into machine code.
    - When using a JIT compiler, the hardware can execute the native code, as opposed to having the JVM interpret the same sequence of bytecode repeatedly and incurring the penalty of a relatively lengthy translation process.
    - This can lead to performance gains in the execution speed unless methods are executed less frequently.



Compiling and Running Java Programs



Difference Between JDK, JRE, and JVM

Parameter	JDK	JRE	JVM
Full-Form	The JDK is an abbreviation for Java Development Kit.	The JRE is an abbreviation for Java Runtime Environment.	The JVM is an abbreviation for Java Virtual Machine.
Definition	The JDK (Java Development Kit) is a software development kit that develops applications in Java. Along with JRE, the JDK also consists of various development tools (Java Debugger, JavaDoc, compilers, etc.)	The Java Runtime Environment (JRE) is an implementation of JVM. It is a type of software package that provides class libraries of Java, JVM, and various other components for running the applications written in Java programming.	The Java Virtual Machine (JVM) is a platform-independent abstract machine that has three notions in the form of specifications. This document describes the requirement of JVM implementation.

Difference Between JDK, JRE, and JVM

Parameter	JDK	JRE	JVM
Functionality	The JDK primarily assists in executing codes. It primarily functions in development.	JRE has a major responsibility for creating an environment for the execution of code.	JVM specifies all of the implementations. It is responsible for providing all of these implementations to the JRE.
Platform Dependency	The JDK is platform-dependent. It means that for every different platform, you require a different JDK.	JRE, just like JDK, is also platform-dependent. It means that for every different platform, you require a different JRE.	The JVM is platform-independent. It means that you won't require a different JVM for every different platform.

Difference Between JDK, JRE, and JVM

Parameter	JDK	JRE	JVM
Tools	Since JDK is primarily responsible for the development, it consists of various tools for debugging, monitoring, and developing java applications.	JRE, on the other hand, does not consist of any tool-like a debugger, compiler, etc. It rather contains various supporting files for JVM, and the class libraries that help JVM in running the program.	JVM does not consist of any tools for software development.
Implementation	**JDK** = Development Tools + JRE (Java Runtime Environment)	**JRE** = Libraries for running the application + JVM (Java Virtual Machine)	**JVM** = Only the runtime environment that helps in executing the Java bytecode.

Difference Between JDK, JRE, and JVM

Parameter	JDK	JRE	JVM
Why Use It?	Why use JDK? Some crucial reasons to use JDK are: •It consists of various tools required for writing Java programs. •JDK also contains JRE for executing Java programs. •It includes an Appletviewer, Java application launcher, compiler, etc. •The compiler helps in converting the code written in Java into bytecodes. •The Java application launcher helps in opening a JRE. It then loads all of the necessary details and then executes all of its main methods.	Why use JRE? Some crucial reasons to use JRE are: •If a user wants to run the Java applets, then they must install JRE on their system. •The JRE consists of class libraries along with JVM and its supporting files. It has no other tools like a compiler or a debugger for Java development. •JRE uses crucial package classes like util, math, awt, lang, and various runtime libraries.	Why use JVM? Some crucial reasons to use JVM are: •It provides its users with a platform-independent way for executing the Java source code. •JVM consists of various tools, libraries, and multiple frameworks. •The JVM also comes with a Just-in-Time (JIT) compiler for converting the Java source code into a low-level machine language. Thus, it ultimately runs faster than any regular application. •Once you run the Java program, you can run JVM on any given platform to save your time.

Difference Between JDK, JRE, and JVM

Parameter	JDK	JRE	JVM
Features	Features of JDK •Here are a few crucial features of JDK: •It has all the features that JRE does. •JDK enables a user to handle multiple extensions in only one catch block. •It basically provides an environment for developing and executing the Java source code. •It has various development tools like the debugger, compiler, etc. •One can use the Diamond operator to specify a generic interface in place of writing the exact one. •Any user can easily install JDK on Unix, Mac, and Windows OS (Operating Systems).	Features of JRE •Here are a few crucial features of JRE: •It is a set of tools that actually helps the JVM to run. •The JRE also consists of deployment technology. It includes Java Plug-in and Java Web Start as well. •A developer can easily run a source code in JRE. But it does not allow them to write and compile the concerned Java program. •JRE also contains various integration libraries like the JDBC (Java Database Connectivity), JNDI (Java Naming and Directory Interface), RMI (Remote Method Invocation), and many more. •It consists of the JVM and virtual machine client for Java HotSpot.	Features of JVM Here are a few crucial features of JVM: •The JVM enables a user to run applications on their device or in a cloud environment. •It helps in converting the bytecode into machine-specific code. •JVM also provides some basic Java functions, such as garbage collection, security, memory management, and many more. •It uses a library along with the files given by JRE (Java Runtime Environment) for running the program. •Both JRE and JDK contain JVM. •It is easily customizable. For instance, a user can feasibly allocate a maximum and minimum memory to it. •JVM can also execute a Java program line by line. It is thus also known as an interpreter. •JVM is also independent of the OS and hardware. It means that once a user writes a Java program, they can easily run it anywhere.

C++ vs Java

There are many differences and similarities between C++ programming language and Java. A list of top differences between C++ and Java are given below:

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in window, web-based, enterprise and mobile applications.
Goto	C++ supports goto statement.	Java doesn't support goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by interfaces in java.
Operator Overloading	C++ supports operator overloading.	Java doesn't support operator overloading.
Pointers	C++ supports pointers. You can write pointer program in C++.	Java supports pointer internally. But you can't write the pointer program in java. It means java has restricted pointer support in java.
Compiler and Interpreter	C++ uses compiler only.	Java uses compiler and interpreter both.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
Thread Support	C++ doesn't have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in thread support.
Documentation comment	C++ doesn't support documentation comment.	Java supports documentation comment (/** ... */) to create documentation for java source code.
Virtual Keyword	C++ supports virtual keyword so that we can decide whether or not override a function.	Java has no virtual keyword. We can override all non-static methods by default. In other words, non-static methods are virtual by default.
unsigned right shift >>>	C++ doesn't support >>> operator.	Java supports unsigned right shift >>> operator that fills zero at the top for the negative numbers. For positive numbers, it works same like >> operator.
Inheritance Tree	C++ creates a new inheritance tree always.	Java uses single inheritance tree always because all classes are the child of Object class in java. Object class is the root of inheritance tree in java.

Path and Classpath variables

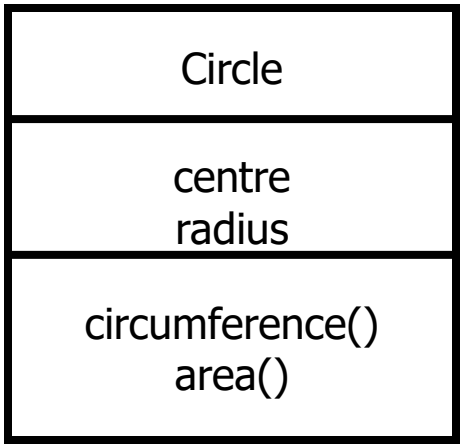
- **What is PATH?**
  - •OS uses the PATH variable to find the set of directories which contains executable files.
  - •PATH setup environment for OS.
  - •To set the PATH in Java, we need to include JDK_HOME/bin directory in PATH environment variable.
  - •PATH can not be overridden by any Java settings.
  - PATH is location of bin files (binary executable files). Example - java.exe, javac.exe.
- **What is CLASSPATH?**
  - CLASSPATH setup environment for Java.
  - CLASSPATH is the location of the .class file, which is created after compiling the java source file.
  - Java classloader (or JVM) uses the CLASSPATH variable to find the location of classes and packages.
  - To set the CLASSPATH, we need to include all those directories where we have put either the .class file or JAR file which is required by the Java application.

Path	Classpath
Path is an environment variable which is used by the operating system to find the executables.	Classpath is an environment variable which is used by the Java compiler to find the path, of classes.ie in J2EE we give the path of jar files.
PATH is nothing but setting up an environment for operating system. Operating System will look in this PATH for executables.	Classpath is nothing but setting up the environment for Java. Java will use to find compiled classes
Path refers to the system	classpath refers to the Developing Envornment
In path we set the path of executables	classpath we set path of jars for compiling classes

- Java is a true OO language and therefore the underlying structure of all Java programs is classes.
- Anything we wish to represent in Java must be encapsulated in a class that defines the “state” and “behaviour” of the basic program components known as objects.
- Classes create objects and objects use methods to communicate between them. They provide a convenient method for packaging a group of logically related data items and functions that work on them.
- A class essentially serves as a template for an object and behaves like a basic data type “int”. It is therefore important to understand how the fields and methods are defined in a class and how they are used to build a Java program that incorporates the basic OO concepts such as encapsulation, inheritance, and polymorphism.

Classes

- A *class* is a collection of *fields* (data) and *methods* (procedure or function) that operate on that data.



Classes

- A *class* is a collection of *fields* (data) and *methods* (procedure or function) that operate on that data.
- The basic syntax for a class definition:

```
class  ClassName [extends
SuperClassName]
{
        [fields declaration]
        [methods declaration]
}
```

- Bare bone class – no fields, no methods

```
public class Circle {
    // my circle class
}
```

Adding Fields: Class Circle with fields

- Add *fields*

```
public class Circle {
        public double x, y;  // centre coordinate
        public double r;     //  radius of the circle

}
```

- The fields (data) are also called the *instance* varaibles.

Adding Methods

- A class with only data fields has no life. Objects created by such a class cannot respond to any messages.
- Methods are declared inside the body of the class but immediately after the declaration of data fields.
- The general form of a method declaration is:

```
type MethodName (parameter-list)
{
        Method-body;
}
```

Adding Methods to Class Circle

```
public class Circle {

    public double x, y; // centre of the circle
    public double r;    // radius of circle

    //Methods to return circumference and area
    public double circumference() {
            return 2*3.14*r;
    }
    public double area() {
            return 3.14 * r * r;
    }
}
```

Method Body

Data Abstraction

- Declare the Circle class, have created a new data type – Data Abstraction

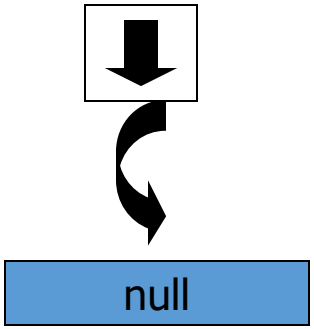
- Can define variables (objects) of that type:

```
Circle  aCircle;
Circle  bCircle;
```

Class of Circle cont.

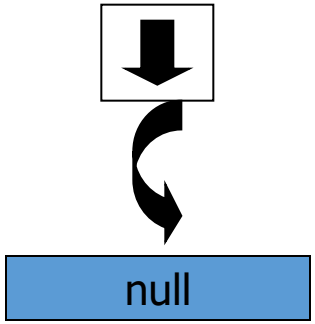
- aCircle, bCircle simply refers to a Circle object, not an object itself.

aCircle



Points to nothing (Null Reference)

bCircle



Points to nothing (Null Reference)

Class and Object

- **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:
    - Object
    - Class
    - Inheritance
    - Polymorphism
    - Abstraction
    - Encapsulation
- An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.
    - It is an instance of a class.
    - An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc.
    - It can be physical or logical (tangible and intangible).
    - The example of an intangible object is the banking system.

Class and Object

- An object has three characteristics:
  - State: represents the data (value) of an object.
  - Behavior: represents the behavior (functionality) of an object such as deposit, withdraw, etc.
  - Identity: An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.
  - For Example, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.
- A **class** is a group of objects which have common properties.
- It is a template or blueprint from which objects are created.
- It is a logical entity. It can't be physical.
- Syntax to declare a class:

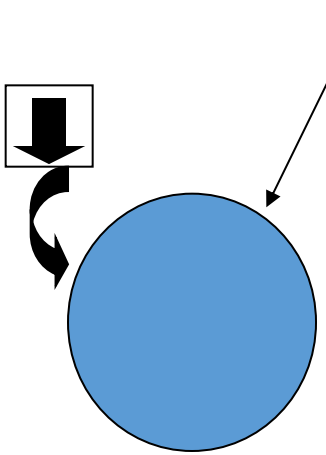
```
class <class_name>
{
        f ield;
        method;  }
```

Illustrate the concepts of Objects

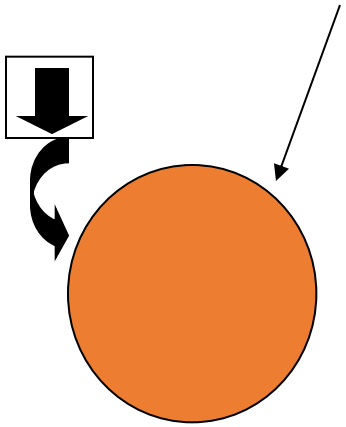
Creating objects of a class

- Objects are created dynamically using the *new* keyword.
- aCircle and bCircle refer to Circle objects

aCircle = new Circle() ;



bCircle = new Circle() ;

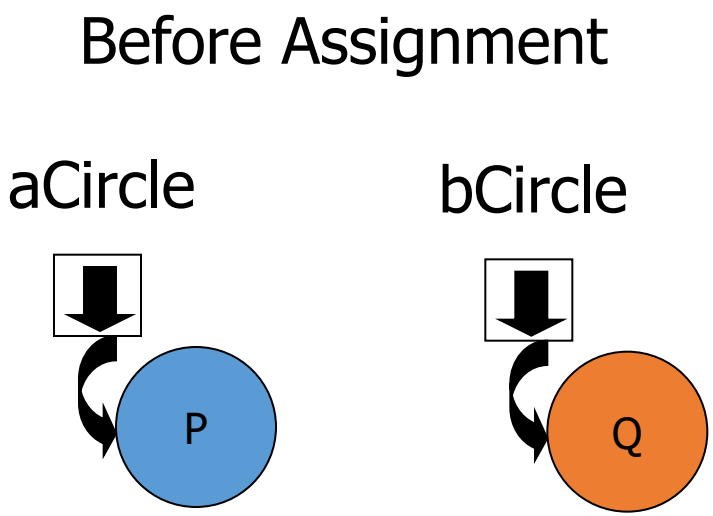


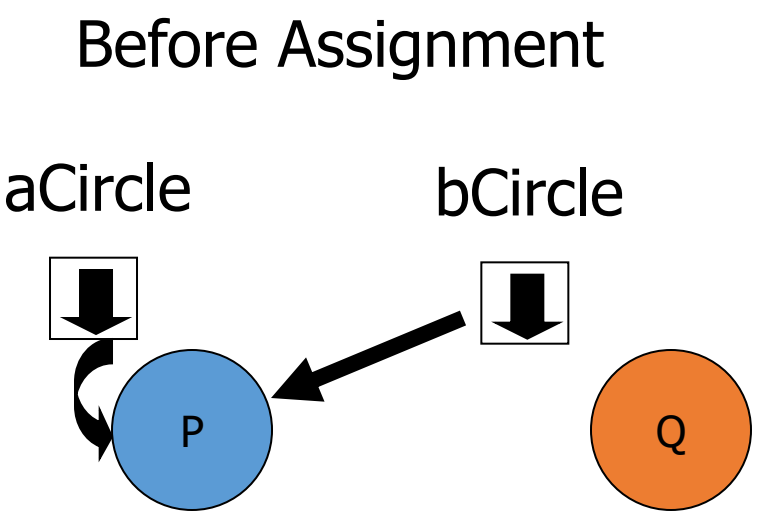
Creating objects of a class

```
aCircle = new Circle();
bCircle = new Circle() ;

bCircle = aCircle;
```

Creating objects of a class

```
aCircle = new Circle();
bCircle = new Circle() ;

bCircle = aCircle;
```





Automatic garbage collection

- The object does not have a reference and cannot be used in future.
- The object becomes a candidate for automatic garbage collection.
- Java automatically collects garbage periodically and releases the memory used to be used in the future.

Accessing Object/Circle Data

- Similar to C syntax for accessing data defined in a structure.

ObjectName.VariableName
ObjectName.MethodName(parameter-list)

```
Circle aCircle = new Circle();

aCircle.x = 2.0 // initialize center and radius
aCircle.y = 2.0
aCircle.r = 1.0
```

Executing Methods in Object/Circle

- Using Object Methods:

sent ‘message’ to aCircle

```
Circle aCircle = new Circle();

double area;
aCircle.r = 1.0;
area = aCircle.area();
```

Using Circle Class

```
// Circle.java:  Contains both Circle class and its user class
//Add Circle class code here
class MyMain
{
    public static void main(String args[])
    {
        Circle aCircle;  // creating reference
        aCircle = new Circle(); // creating object
        aCircle.x = 10;  // assigning value to data field
        aCircle.y = 20;
        aCircle.r = 5;
        double area = aCircle.area(); // invoking method
        double circumf = aCircle.circumference();
        System.out.println("Radius="+aCircle.r+" Area="+area);
        System.out.println("Radius="+aCircle.r+" Circumference ="+circumf);
    }
}
```

Example

```
class Animal {
public void move()
{     System.out.println("Animals can move");
}}
class Dog extends Animal {
public void move() {
System.out.println("Dogs can walk and run");
 }}
public class TestDog {
public static void main(String args[]) {
Animal a = new Animal(); // Animal reference and object
Animal b = new Dog();   // Animal reference but Dog object
a.move();   // runs the method in Animal class
b.move();   // runs the method in Dog class
}}
```

Class and Object

We can create a class in Java using the **class** keyword. For example,

```
class ClassName
 {
 // fields
// methods
}
```

- Here, fields (variables) and methods represent the state and behavior of the object respectively.
- Fields are used to store data methods are used to perform some operations

Class and Object

- **Instance variable in Java**
  - A variable which is created inside the class but outside the method is known as an instance variable.
  - Instance variable doesn't get memory at compile time.
  - It gets memory at runtime when an object or instance is created.
  - That is why it is known as an instance variable.
- **Method in Java**
  - A method is like a function which is used to expose the behavior of an object.
  - Advantage of Method
    - Code Reusability and Code Optimization

Class and Object

- **new keyword in Java:**
  - The new keyword is used to allocate memory at runtime.
  - All objects get memory in Heap memory area.
- **Ways to initialize object:**
  - There are 3 ways to initialize object in Java.
    - By reference variable
    - By method
    - By constructor

Class and Object

Initialize object by reference variable

File: TestStudent2.java

```
class Student{
 int id;
 String name;
}
class TestStudent2{
 public static void main(String args[]){
  Student s1=new Student();
   s1.id=101;
   s1.name="Sonoo";
   System.out.println(s1.id+" "+s1.name);//printing members with a white space
 }
 }
```

Output:

```
101 Sonoo
```

We can also create multiple objects and store information in it through reference variable.

Class and Object

Initialize object by method

File: TestStudent4.java

```
class Student{
 int rollno;
 String name;
 void insertRecord(int r, String n){
  rollno=r;
  name=n;
 }
 void displayInformation(){System.out.println(rollno+" "+name);}
}
class TestStudent4{
 public static void main(String args[]){
  Student s1=new Student();
  Student s2=new Student();

   s1.displayInformation();

    s2.displayInformation();

   }
```

Class and Object

Initialize object by reference variable

```
// Java Program to demonstrate
// Constructor
import java.io.*;

// Driver Class
class  Cons {

    // Constructor
    Cons()
    {
        super();
        System.out.println("Constructor Called");
    }

    // main function
    public static void main(String[] args)
    {
        Cons Cons = new Cons();
    }
}
```

Explain the concepts of Abstract classes

What is an Abstract class?

- Data **abstraction** is the process of hiding certain details and showing only essential information to the user.
- Abstraction can be achieved with either **abstract classes** or **interfaces.**

The `abstract` keyword is a non-access modifier, used for classes and methods:

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).

- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

What is an Abstract class?

- An abstract class can have both abstract and regular methods:

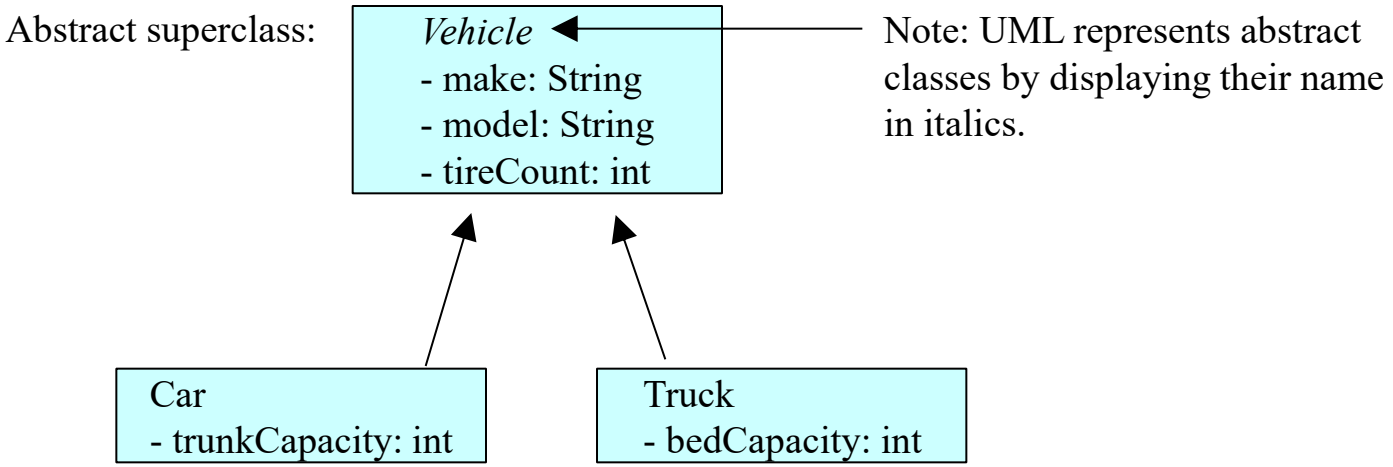
```
abstract class Animal
{
    public abstract void
    animalSound(); public void sleep()
    { System.out.println("Zzz"); }

 }
```

From the example above, it is not possible to create an object of the Animal class:

```
Animal myObj = new Animal(); // will generate an error
```

Abstract Class Example

- In the following example, the subclasses represent objects taken from the problem domain.
- The superclass represents an abstract concept that does not exist "as is" in the real world.



What Are Abstract Classes Used For?

- Abstract classes are used heavily in Design Patterns
  - Creational Patterns: Abstract class provides interface for creating objects. The subclasses do the actual object creation
  - Structural Patterns: How objects are structured is handled by an abstract class. What the objects do is handled by the subclasses
  - Behavioural Patterns: Behavioural interface is declared in an abstract superclass. Implementation of the interface is provided by subclasses.
- Be careful not to over use abstract classes
  - Every abstract class increases the complexity of your design
  - Every subclass increases the complexity of your design
  - Ensure that you receive acceptable return in terms of functionality given the added complexity.

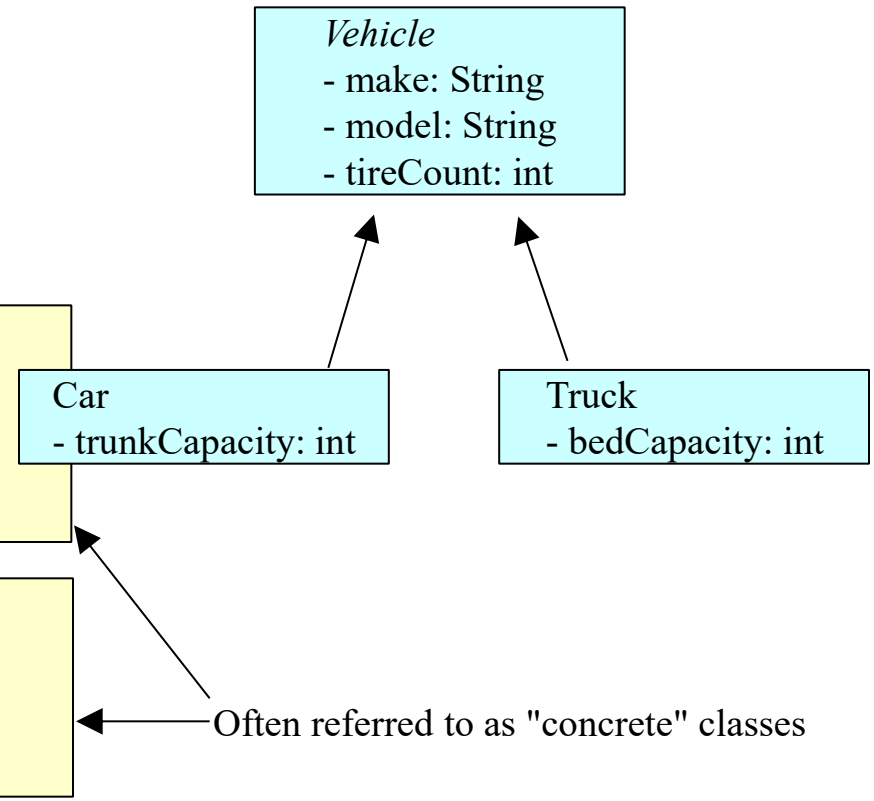
Defining Abstract Classes

- Inheritance is declared using the "extends" keyword
  - If inheritance is not defined, the class extends a class called Object

```
public abstract class Vehicle
{
  private String make;
  private String model;
  private int tireCount;
  [...]
```

```
public class Car extends Vehicle
{
  private int trunkCapacity;
  [...]
```

```
public class Truck extends Vehicle
{
  private int bedCapacity;
  [...]
```

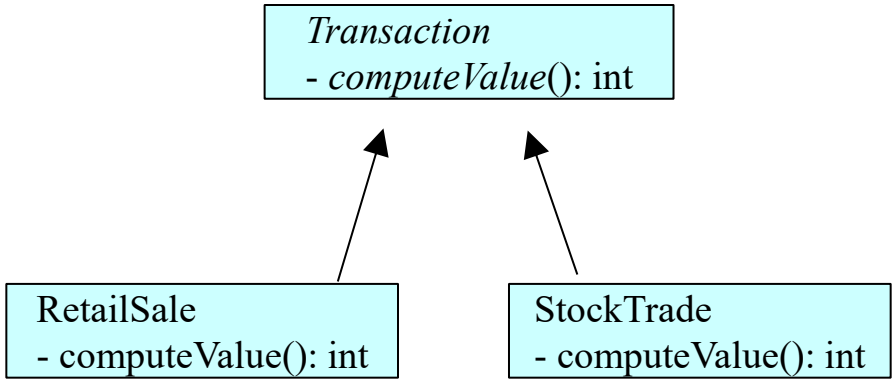


Abstract Methods

- Methods can also be abstracted
  - An abstract method is one to which a signature has been provided, but no implementation for that method is given.
  - An Abstract method is a placeholder. It means that we declare that a method must exist, but there is no meaningful implementation for that methods within this class
- Any class which contains an abstract method MUST also be abstract
  - Any class which has an incomplete method definition cannot be instantiated (ie. it is abstract)
- Abstract classes can contain both concrete and abstract methods.
  - If a method can be implemented within an abstract class, and implementation should be provided.

Abstract Method Example

- In the following example, a Transaction's value can be computed, but there is no meaningful implementation that can be defined within the Transaction class.
  - How a transaction is computed is dependent on the transaction's type
  - *Note: This is polymorphism.*



Defining Abstract Methods

- Inheritance is declared using the "extends" keyword
  - If inheritance is not defined, the class extends a class called Object

```
public abstract class Transaction
{
  public abstract int computeValue();
```

Note: no implementation

Transaction
- *computeValue*(): int

```
public class RetailSale extends Transaction
{
  public int computeValue()
  {
        [...]
```

RetailSale
- computeValue(): int

StockTrade
- computeValue(): int

```
public class StockTrade extends Transaction
{
  public int computeValue()
  {
        [...]
```

Exercise

Q) A university wants to develop a **Payment Processing System** that supports multiple payment modes such as **Credit Card** and **UPI**.

Write a Java Program (Requirements):

1. Create an interface **Payment** with the method: **processPayment**
2. Implement two classes:
   - **CreditCardPayment**
   - **UPIPayment**
     Each class should provide its own implementation of **processPayment()**.
3. Create a class **PaymentService** that:
   - Accepts a **Payment** object as input.
   - Calls the **processPayment()** method to complete the transaction.
4. Demonstrate **runtime polymorphism** by processing payments using different payment modes.

Describe the concepts of Static classes and Inner classes

What is static

- The static keyword is used when a member variable of a class has to be shared between all the instances of the class.
- All static variables and methods belong to the class and not to any instance of the class
- The static keyword in Java is used for memory management mainly.
- We can apply java static keyword with variables, methods, blocks and nested class .
- The static keyword belongs to the class than an instance of the class.
- When a class is loaded by the virtual machine all the static variables and methods are available for use.
- Hence we don’t need to create any instance of the class for using the static variables or methods.
- Variables which don’t have static keyword in the definition are implicitly non static.

Program of the counter without static variable

```
class Counter{
int count=0;//will get memory each time when the instance is created

Counter(){
count++;//incrementing value
System.out.println(count);
}

public static void main(String args[]){
//Creating objects
Counter c1=new Counter();
Counter c2=new Counter();
Counter c3=new Counter();
}
}
```

Output:

1
1
1

Program of counter by static variable

```
class Counter2{
static int count=0;//will get memory only once and retain its value

Counter2(){
count++;//incrementing the value of static variable
System.out.println(count);
}

public static void main(String args[]){
//creating objects
Counter2 c1=new Counter2();
Counter2 c2=new Counter2();
Counter2 c3=new Counter2();
}
}
```

Output:

1
2
3

Static Methods

- If we apply static keyword with any method, it is known as static method.
- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.
- Java program to call a static method

```
class Test {
// static method
public static int sum(int a, int b)
{
return a + b;
} }
```

Static methods

```
class Main {
   public static void main(String[] args)
   {
      int n = 3, m = 6;

      // call the static method
      int s = Test.sum(n, m);

      System.out.print("sum is = " + s);
   }
}
```

Output:
sum is = 9

Example

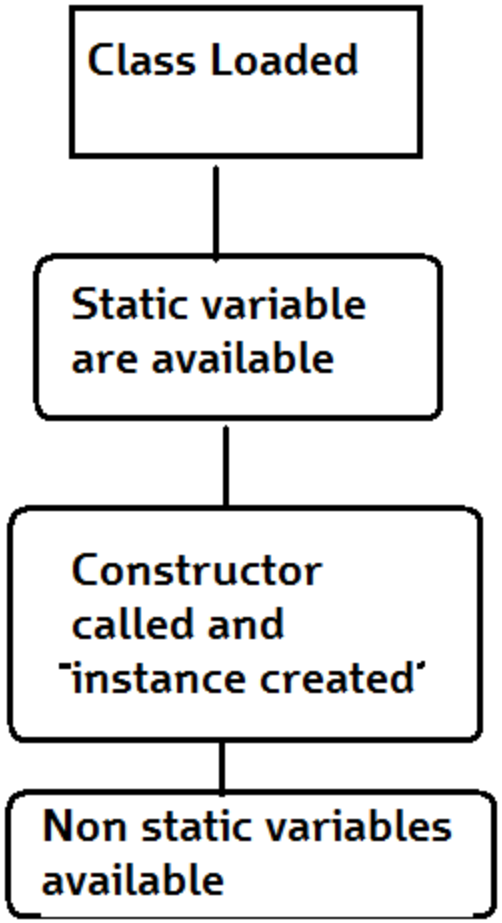
```
Class staticDemo{
    public static int a = 100; // All instances of staticDemo have this variable as a common `
                                        variable
    public int b =2 ;
    public static showA(){
            System.out.println("A is "+a);
    }
}
Class execClass{
    public static void main(String args[]){
            staticDemo.a = 35; // when we use the class name, the class is loaded, direct access to a
    without any instance
            staticDemo.b=22; // ERROR this is not valid for non static variable
            staticDemo demo = new staticDemo();
            demo.b = 200;  // valid to set a value for a non static variable after creating an instance.
            staticDemo.showA();  //prints 35
    }
}
```

Static and Non-static

- We can access static variables without creating an instance of the class
- As they are already available at class loading time, we can use them in any of our non static methods.
- We cannot use non static methods and variables without creating an instance of the class as they are bound to the instance of the class.
- They are initialized by the constructor when we create the object using new operator.

STATIC VARIABLE	NON STATIC VARIABLE
Static variables can be accessed using class name	Non static variables can be accessed using instance of a class
Static variables can be accessed by static and non static methods	Non static variables cannot be accessed inside a static method.
Static variables reduce the amount of memory used by a program.	Non static variables do not reduce the amount of memory used by a program
Static variables are shared among all instances of a class.	Non static variables are specific to that instance of a class.
Static variable is like a global variable and is available to all methods.	Non static variable is like a local variable and they can be accessed through only instance of a class

How it works



Basic Steps of how objects are created

1. Class is loaded by JVM
2. Static variable and methods are loaded and initialized and available for use
3. Constructor is called to instantiate the non static variables
4. Non static variables and methods are now available

- As all the non static variable are available only after the constructor is called, there is a restriction on using non static variable in static methods.

Why do we need this

- Static methods are identified to be mostly used when we are writing any utility methods.
- We can also use static variables when sharing data.
- When sharing data do keep in mind about multithreading can cause inconsistency in the value. (synchronize the variable)

Inner Classes

- **Inner classes** are classes defined within other classes
  - The class that includes the inner class is called the **outer class**
  - There is no particular location where the definition of the inner class (or classes) must be place within the outer class
  - Placing it first or last, however, will guarantee that it is easy to find

Simple Uses of Inner Classes

- An inner class definition is a member of the outer class in the same way that the instance variables and methods of the outer class are members
  - An inner class is local to the outer class definition
  - The name of an inner class may be reused for something else outside the outer class definition
  - If the inner class is private, then the inner class cannot be accessed by name outside the definition of the outer class

Inner/Outer Classes

```
public class Outer
{
        private class Inner
        {
                // inner class instance variables
                // inner class methods
        } // end of inner class definition
        // outer class instance variables
        // outer class methods
}
```

Simple Uses of Inner Classes

- There are two main advantages to inner classes
  - They can make the outer class more self-contained since they are defined inside a class
  - Both of their methods have access to each other's private methods and instance variables

Interface

- An interface is a contract that specifies a set of methods without providing their implementations.
- It acts as a blueprint for classes that implement it.
- Interfaces enable multiple classes to share a common set of methods without forcing them into a specific inheritance hierarchy.
- Interfaces are completely abstract and can't have method implementations
- No constructors: Interfaces can't have constructors because they can't be instantiated
- Public, static, and final: All attributes of an interface are public, static, and final by default
- Implicitly abstract and public: Interface methods are implicitly abstract and public by default.
- Extending: An interface can't extend a class, but it can extend another interface.
- Implementing: Classes use the implements keyword to implement an interface.
- Functional interfaces: A functional interface has a single abstract method. The Function interface is an example of a functional interface.

Interface

- Marker interfaces: A marker interface, also known as a tagging interface, doesn't have any methods or variables. It's used to mark a class so that it can be identified by the JVM or other tools.
- List interface: The List interface is a child interface of Collection. Some classes that implement the List interface include LinkedList, ArrayList, Vector, and Stack.
- So we need an Interface in Java for the following reasons:
    - Total Abstraction : An interface only stores the method signature and not the method definition.
    - Multiple Inheritance: Without Interface, the process of multiple inheritances is impossible as the conventional way of inheriting multiple parent classes results in profound ambiguity. This type of ambiguity is known as the Diamond problem. Interface resolves this issue.
    - Loose-Coupling: The term Coupling describes the dependency of one class for the other. So, while using an interface, we define the method separately and the signature separately. This way, all the methods, and classes are entirely independent and archives Loose Coupling.

Interface

- Marker interfaces: A marker interface, also known as a tagging interface, doesn't have any methods or variables. It's used to mark a class so that it can be identified by the JVM or other tools.
- List interface: The List interface is a child interface of Collection. Some classes that implement the List interface include LinkedList, ArrayList, Vector, and Stack.

- set of methods without forcing them into a specific inheritance hierarchy.

Defining an Interface

In Java, you define an interface using the interface keyword:

```
public interface Shape {
double area();
void draw();
}
```

Implementing an Interface

A class implements an interface by using the **implements** keyword. It must provide implementations for all the…

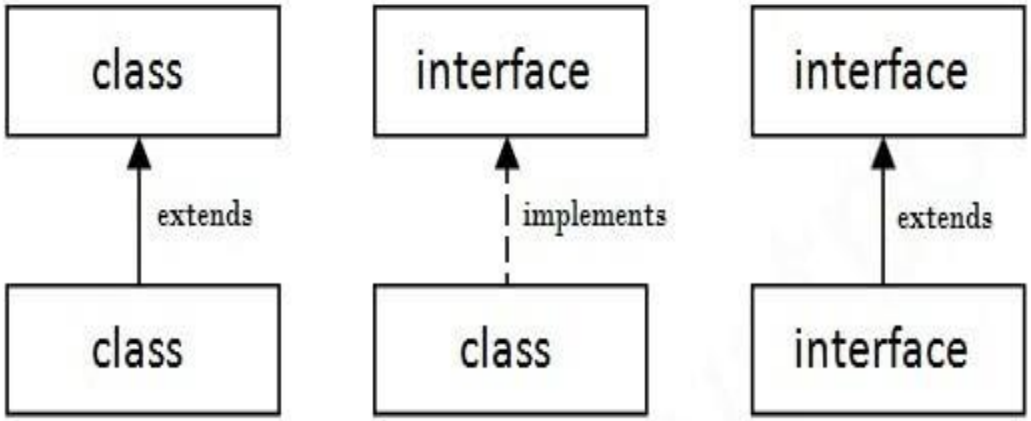
Interface

```
interface FirstInterface {
public void myMethod(); // interface method
}
interface SecondInterface {
public void myOtherMethod(); // interface method
}
// DemoClass "implements" FirstInterface and SecondInterface
class DemoClass implements FirstInterface, SecondInterface {
public void myMethod() { System.out.println("Some text..");
} public void myOtherMethod() { System.out.println("Some
other text..."); } }
class MyMainClass {
public static void main(String[] args) {
 DemoClass myObj = new DemoClass();
myObj.myMethod();
myObj.myOtherMethod();
} }
```

Interface

The relationship between classes and interfaces

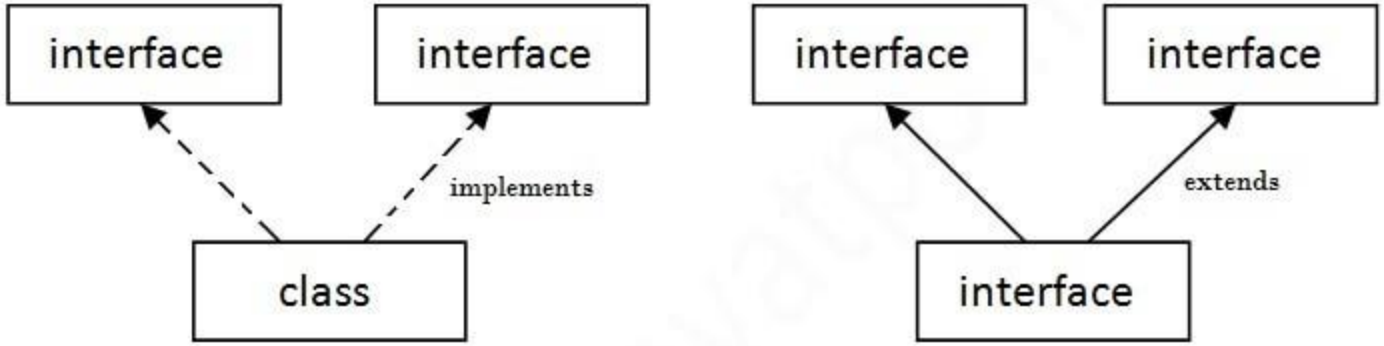
As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



Interface

Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

Interface

```
interface printable{
void print();
}
class A6 implements printable{
public void print(){System.out.println("Hello");}

public static void main(String args[]){
A6 obj = new A6();
obj.print();
 }
}
```

Interface

```
interface Printable{
void print();
}
interface Showable{
void show();
}
class A7 implements Printable,Showable{
public void print(){System.out.println("Hello");}
public void show(){System.out.println("Welcome");}

public static void main(String args[]){
A7 obj = new A7();
obj.print();
obj.show();
 }
}
```

Illustrate the Data types and variable

Data type

- Data type specifies the size and type of values that can be stored in an identifier. The Java language is rich in its data types. Different data types allow you to select the type appropriate to the needs of the application.
- Data types in Java are classified into two types:
- Primitive—which include Integer, Character, Boolean, and Floating Point.
- Non-primitive—which include Classes, Interfaces, and Arrays.

Primitive Data Types

1. Integer

- Integer types can hold whole numbers such as 123 and −96. The size of the values that can be stored depends on the integer type that we choose.

Type	Size	Range of values that can be stored
byte	1 byte	−128 to 127
short	2 bytes	−32768 to 32767
int	4 bytes	−2,147,483,648 to 2,147,483,647
long	8 bytes	9,223,372,036,854,775,808 to 9,223,372,036,854,755,807

- **2. Floating Point**
- Floating point data types are used to represent numbers with a fractional part. Single precision floating point numbers occupy 4 bytes and Double precision floating point numbers occupy 8 bytes. There are two subtypes:

Type	Size	Range of values that can be stored
float	4 bytes	3.4e−038 to 3.4e+038
double	8 bytes	1.7e−308 to 1.7e+038

- **3. Character**
- It stores character constants in the memory. It assumes a size of 2 bytes, but basically it can hold only a single character because char stores unicode character sets. It has a minimum value of ‘u0000’ (or 0) and a maximum value of ‘uffff’ (or 65,535, inclusive).
- **4. Boolean**
- Boolean data types are used to store values with two states: true or false.

- **Java Tokens**
- A token is the smallest element in a program that is meaningful to the compiler. These tokens define the structure of the language. The Java token set can be divided into five categories: Identifiers, Keywords, Literals, Operators, and Separators.
- **1. Identifiers**
- Identifiers are names provided by you. These can be assigned to variables, methods, functions, classes etc. to uniquely identify them to the compiler.
- **2. Keywords**
- Keywords are reserved words that have a specific meaning for the compiler. They cannot be used as identifiers. Java has a rich set of keywords. Some examples are: boolean, char, if, protected, new, this, try, catch, null, threadsafe etc.

3. Literals

Literals are variables whose values remain constant throughout the program. They are also called Constants. Literals can be of four types. They are:

a. String Literals

String Literals are always enclosed in double quotes and are implemented using the java.lang.String class. Enclosing a character string within double quotes will automatically create a new String object. For example, String s = "this is a string";. String objects are immutable, which means that once created, their values cannot be changed.

b. Character Literals

These are enclosed in single quotes and contain only one character.

c. Boolean Literals

They can only have the values true or false. These values do not correspond to 1 or 0 as in C or C++.

d. Numeric Literals

Numeric Literals can contain integer or floating point values.

4. Operators

An operator is a symbol that operates on one or more operands to produce a result.

They will be discussed in greater detail in the next article.

5. Separators

Separators are symbols that indicate the division and arrangement of groups of code. The structure and function of code is generally defined by the separators. The separators used in Java are as follows:

parentheses ( )

Used to define precedence in expressions, to enclose parameters in method definitions, and enclosing cast types.

braces { }

Used to define a block of code and to hold the values of arrays.

brackets [ ]

Used to declare array types.

semicolon ;

Used to separate statements.

comma ,

Used to separate identifiers in a variable declaration and in the for statement.

period .

Used to separate package names from classes and subclasses and to separate a variable or a method from a reference variable.

Variables

There are different types of variables in Java. They are as follows:

1. Instance Variables (Non-Static Fields)

Objects store their individual states in “non-static fields”, that is, fields declared without the static keyword.

Non-static fields are also known as instance variables because their values are unique to each instance of a class. For example, the currentSpeed of one bicycle is independent from the currentSpeed of another.

2. Class Variables (Static Fields)

A class variable is any field declared with the static modifier; this tells the compiler that there is exactly one copy of this variable in existence, regardless of how many times the class has been instantiated. A field defining the number of gears for a particular kind of bicycle could be marked as static since, conceptually, the same number of gears will apply to all instances. The code static int numGears = 6; would create such a static field.

3. Local Variables

A method stores its temporary state in local variables. The syntax for declaring a local variable is similar to declaring a field (for example, int count = 0;). There is no special keyword designating a variable as local; that determination comes entirely from the location in which the variable is declared—between the opening and closing braces of a method. As such, local variables are only visible to the methods in which they are declared; they are not accessible from the rest of the class.

4. Parameters

They are the variables that are passed to the methods of a class

Variable Declaration

Identifiers are the names of variables. They must be composed of only letters, numbers, the underscore, and the dollar sign ($). They cannot contain white spaces. Identifiers may only begin with a letter, the underscore, or the dollar sign. A variable cannot begin with a number. All variable names are case sensitive.

Syntax for variable declaration

datatype1 variable1, datatype2 variable2, … datatypen variablen;

For example:

int a, char ch;

Initialisation

Variables can be assigned values in the following way: Variablename = value;

For example;

ch='a';

a=0;

Expressions

- **Expressions** are essential building blocks of any **Java** program, usually created to produce a new value, although sometimes an **expression** simply assigns a value to a variable.
- **Expressions** are built using values, variables, operators and method calls.
- An *expression* is a construct made up of variables, operators, and method invocations, which are constructed according to the syntax of the language, that evaluates to a single value. You've already seen examples of expressions, illustrated in bold below:

```
int cadence = 0;
anArray[0] = 100;
System.out.println("Element 1 at index 0: " + anArray[0]);

int result = 1 + 2; // result is now 3
if (value1 == value2)
   System.out.println("value1 == value2");
```

- For example, the following expression gives different results, depending on whether you perform the addition or the division operation first:

```
x + y / 100 // ambiguous
```

- You can specify exactly how an expression will be evaluated using balanced parenthesis: (and).

- For example, to make the previous expression unambiguous, you could write the following:

```
(x + y) / 100 // unambiguous, recommended
```

- Operators that have a higher precedence get evaluated first. For example, the division operator has a higher precedence than does the addition operator. Therefore, the following two statements are equivalent:

```
x + y / 100
x + (y / 100) // unambiguous, recommended
```

- When writing compound expressions, be explicit and indicate with parentheses which operators should be evaluated first. This practice makes code easier to read and to maintain.

Describe the concepts of operators in any problem

Content

- Group of Operators
- Arithmetic Operators
- Assignment Operator
- Order of Precedence
- Increment/Decrement Operators
- Relational Operators
- Logical Operators

Operators

- Operators are special symbols used for:
  - mathematical functions
  - assignment statements
  - logical comparisons
- Examples of operators:
  - 3 + 5 // uses + operator
  - 14 + 5 – 4 * (5 – 3) // uses +, -, * operators
- Expressions: can be combinations of variables and operators that result in a value

Groups of Operators

- There are 5 different groups of operators:
  - Arithmetic Operators
  - Assignment Operator
  - Increment / Decrement Operators
  - Relational Operators
  - Logical Operators

Java Arithmetic Operators

Addition	+
Subtraction	–
Multiplication	*
Division	/
Remainder (modulus)	%

Arithmetic Operators

- The following table summarizes the arithmetic operators available in Java.

Operation	Java Operator	Example	Value (x = 10, y = 7, z = 2.5)
Addition	+	x + y	17
Subtraction	-	x - y	3
Multiplication	*	x * y	70
Division	/	x / y	1
		x / z	4.0
Modulo division (remainder)	%	x % y	3

This is an integer division where the fractional part is truncated.

Example

Example of division issues:

10 / 3 gives 3

10.0 / 3 gives 3.33333

As we can see,

- if we divide two integers we get an integer result.
- if one or both operands is a floating-point value we get a floating-point result.

Modulus

- Generates the remainder when you divide two integer values.

5%3 gives 2 5%4 gives 1

5%5 gives 0 5%10 gives 5

- Modulus operator is most commonly used with integer operands. If we attempt to use the modulus operator on floating-point values we will garbage!

Order of Precedence

( ) evaluated first, inside-out

*, /, or % evaluated second, left-to-right

+, – evaluated last, left-to-right

Basic Assignment Operator

- We assign a value to a variable using the basic *assignment operator (=)*.
- Assignment operator stores a value in memory.
- The syntax is

```
leftSide = rightSide ;
```

Allways it is a *variable identifier.*

It is either a *literal* | a *variable identifier* | an *expression.*

Examples:

```
i = 1;
start = i;
sum = firstNumber + secondNumber;
avg = (one + two + three) / 3;
```

The Right Side of the Assignment Operator

- The Java assignment operator assigns the value on the right side of the operator to the variable appearing on the left side of the operator.
- The right side may be either:
  - Literal: ex. `i = 1;`
  - Variable identifier: ex. `start = i;`
  - Expression: ex. `sum = first + second;`

Assigning Literals

- In this case, the literal is stored in the space memory allocated for the variable at the left side.

A

```
int firstNumber=1, secondNumber;
firstNumber  = 234;
secondNumber = 87;
```

B

Code

A. Variables are allocated in memory.

firstNumber	1
secondNumber	???

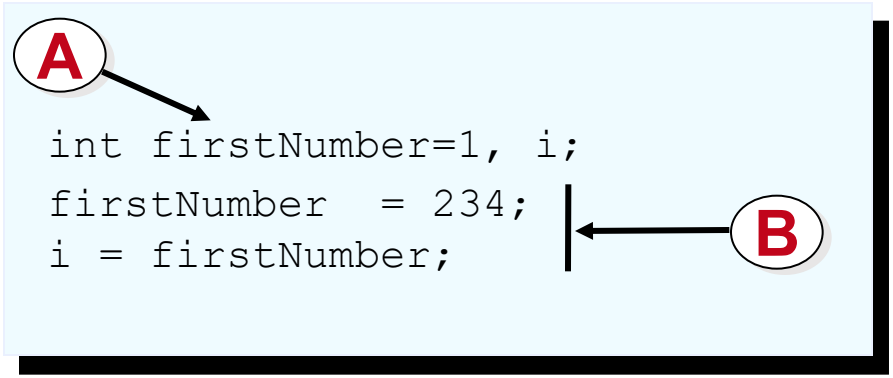
B. Literals are assigned to variables.

firstNumber	234
secondNumber	87

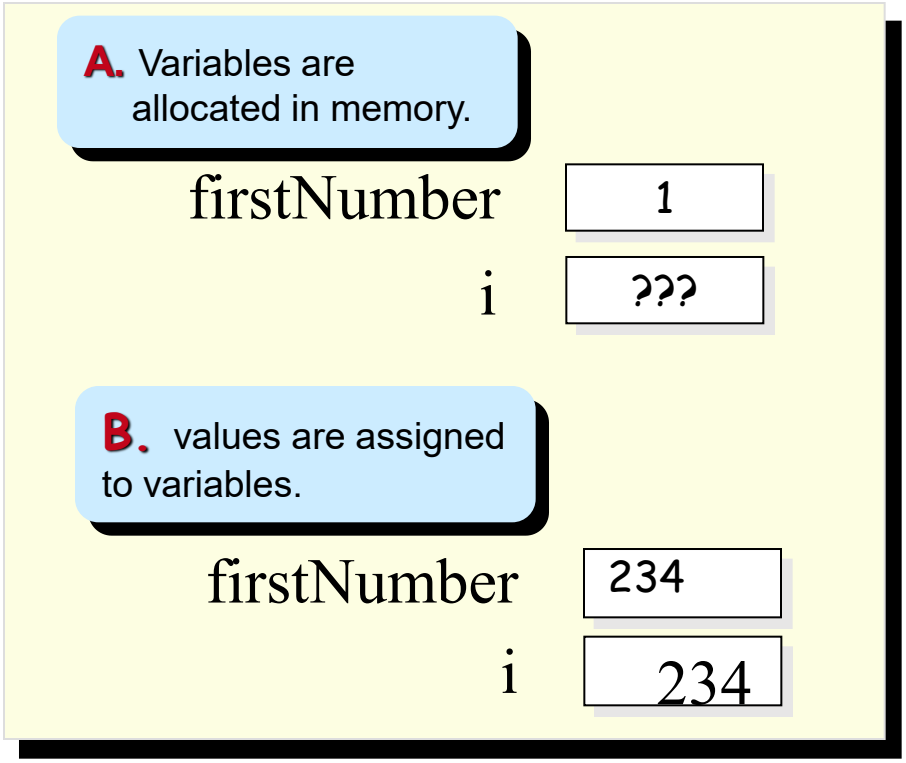
State of Memory

Assigning Variables

- In this case, the value of the variable at the right side is stored in the space memory allocated for the variable at the left side.



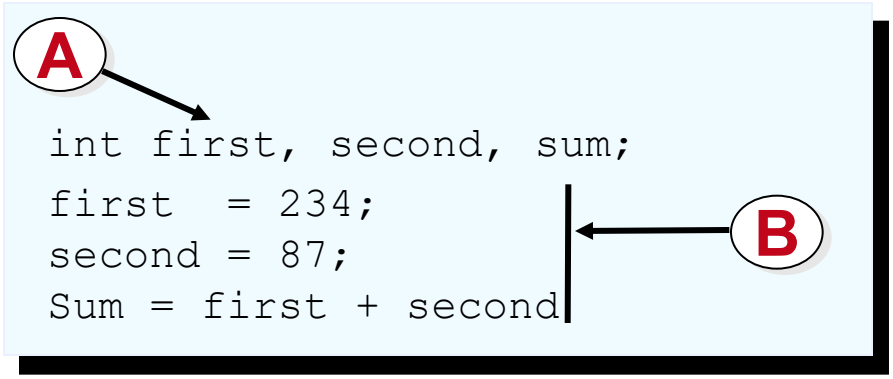
Code



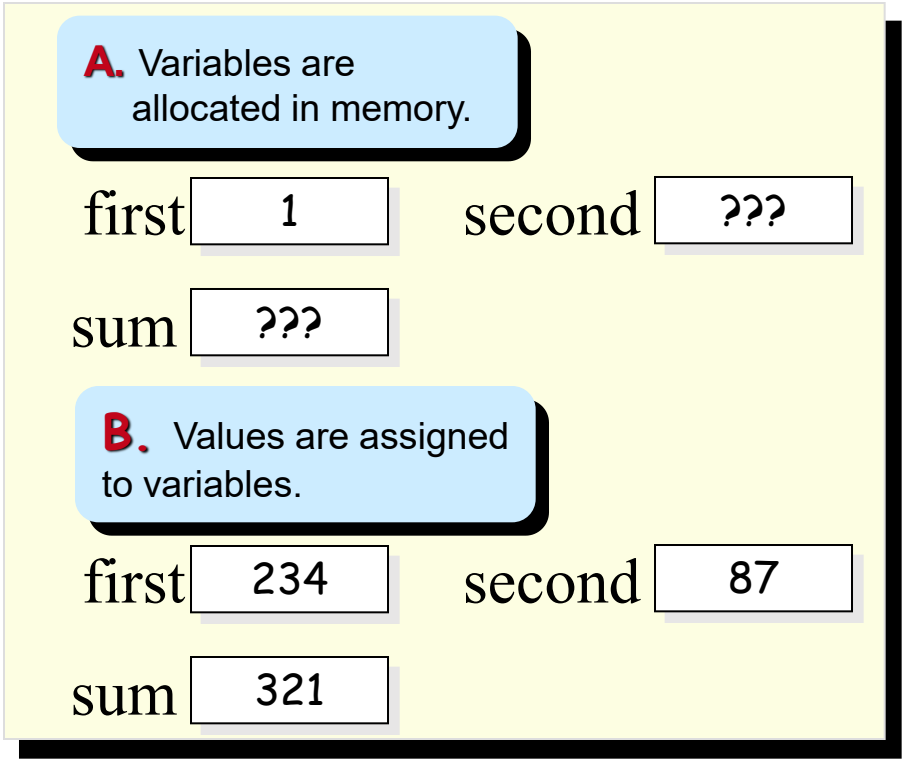
State of Memory

Assigning Expressions

- In this case, the result of the evaluation of the expression is stored in the space memory allocated for variable at the left side.



Code



State of Memory

Updating Data

```
int number;
number = 237;
number = 35;
```

A

B

C

Code

A. The variable is allocated in memory.

number ???

B. The value **237** is assigned to **number**.

number 237

C. The value **35** overwrites the previous value **237**.

number 35

State of Memory

Example: Sum of two integer

```
public class Sum {

 // main method
    public static void main( String args[] ){
         int a, b, sum;
         a = 20;
         b = 10;
         sum = a + b;
         System.out.println(a + ″ + ″ + b + ″  = ″ + sum);

    } // end main

} // end class Sum
```

Arithmetic/Assignment Operators

Java allows combining arithmetic and assignment operators into a single operator:

Addition/assignment	**+=**
Subtraction/assignment	**–=**
Multiplication/assignment	***=**
Division/assignment	**/=**
Remainder/assignment	**%=**

Arithmetic/Assignment Operators

- The syntax is

```
leftSide  Op= rightSide ;
```

It is either a *literal* | a *variable identifier* | an *expression.*

Allways it is a *variable identifier.*

It is an *arithmetic operator.*

- This is equivalent to:

```
leftSide = leftSide Op rightSide ;
```

- x%=5; ⇔ x = x % 5;
- x*=y+w*z; ⇔ x = x*(y+w*z);

Increment/Decrement Operators

Only use ++ or – – when a variable is being incremented/decremented as a statement by itself.

`x++;` is equivalent to `x = x+1;`

`x--;` is equivalent to `x = x-1;`

Relational Operators

- Relational operators compare two values
- They Produce a *boolean* value (**true** or **false)** depending on the relationship

Operation	Is true when
a >b	**a** is greater than **b**
a >=b	**a** is greater than or equal to **b**
a ==b	a is equal to **b**
a !=b	**a** is not equal to **b**
a <=b	**a** is less than or equal to **b**
a <b	**a** is less than **b**

Example

- int x = 3;
- int y = 5;
- boolean result;

  result = (x > y);
- now result is assigned the value false because 3 is not greater than 5

Logical Operators

Symbol	Name
&&	AND
||	OR
!	NOT

&&	T	F
T	T	F
F	F	F

||	T	F
T	T	T
F	T	F

Example

```
boolean x = true;
boolean y = false;
boolean result;

result = (x && y);
```

result is assigned the value false

```
result = ((x || y) && x);
```

(x || y) evaluates to true

(true && x) evaluates to true

result is then assigned the value true

Operators Precedence

Parentheses	(), inside-out
Increment/decrement	++, --, from left to right
Multiplicative	*, /, %, from left to right
Additive	+, -, from left to right
Relational	<, >, <=, >=, from left to right
Equality	==, !=, from left to right
Logical AND	&&
Logical OR	||
Assignment	=, +=, -=, *=, /=, %=

Explain the concepts of control structures in any problem

What are control structures?

- Our programs so far consist of just a list of commands to be done in order
  - The program cannot choose whether or not to perform a command
  - The program cannot perform the same command more than once
  - Such programs are extremely limited!
- Control structures allow a program to base its behavior on the values of variables

boolean

- boolean is one of the eight primitive types
  - booleans are used to make yes/no decisions
  - All control structures use booleans
- There are exactly two boolean values, true ("yes") and false ("no")
  - boolean, true, and false are all lowercase
- booleans are named after George Boole, the founder of Boolean logic

Declaring **boolean** variables

- boolean variables are declared like any other kind of variable:

```
boolean hungry;
boolean passingGrade;
boolean taskCompleted = false;
```

- boolean values can be assigned to boolean variables:

```
taskCompleted = true;
```

Numeric comparisons

- The following numeric comparisons each give a boolean result:

```
x < y     // is x less than y?
x <= y    // is x less than or equal to y?
x == y    // is x equal to y? (do not use =)
x != y    // is x unequal to y?
x >= y    // is x greater than or equal to y?
x > y     // is x greater than y?
```

- Reminder: Don't use == or != for floating-point numbers

The **if** statement

- The if statement has the form:

  if (*boolean-expression*) *statement*

- Examples:

```
if (passingGrade) System.out.println("Whew!");
if (x > largest) largest = x;
if (citBook.price < 40.00) citBook.purchase();
```

- The if statement controls *one* other statement
  - Often this isn't enough; we want to control a *group* of statements

Compound statements

- We can use braces to group together several statements into one "compound" statement:

  { *statement*; *statement*; ...; *statement*; }

- Braces can group any number of statements:

```
{ }          // OK--this is an "empty" statement
{ x = 0; }   // OK--braces don't hurt
{ temp = x;
  x = y;
  y = temp; }   //typical use
```

- The compound statement is the only kind of statement that does *not* end with a semicolon

The if statement again

- The if statement controls *one* other statement, but it can be a compound statement
- Example:

```
if (cost < amountInPocket) {
    System.out.println("Spending $" + cost);
    amountInPocket = amountInPocket - cost;
}
```

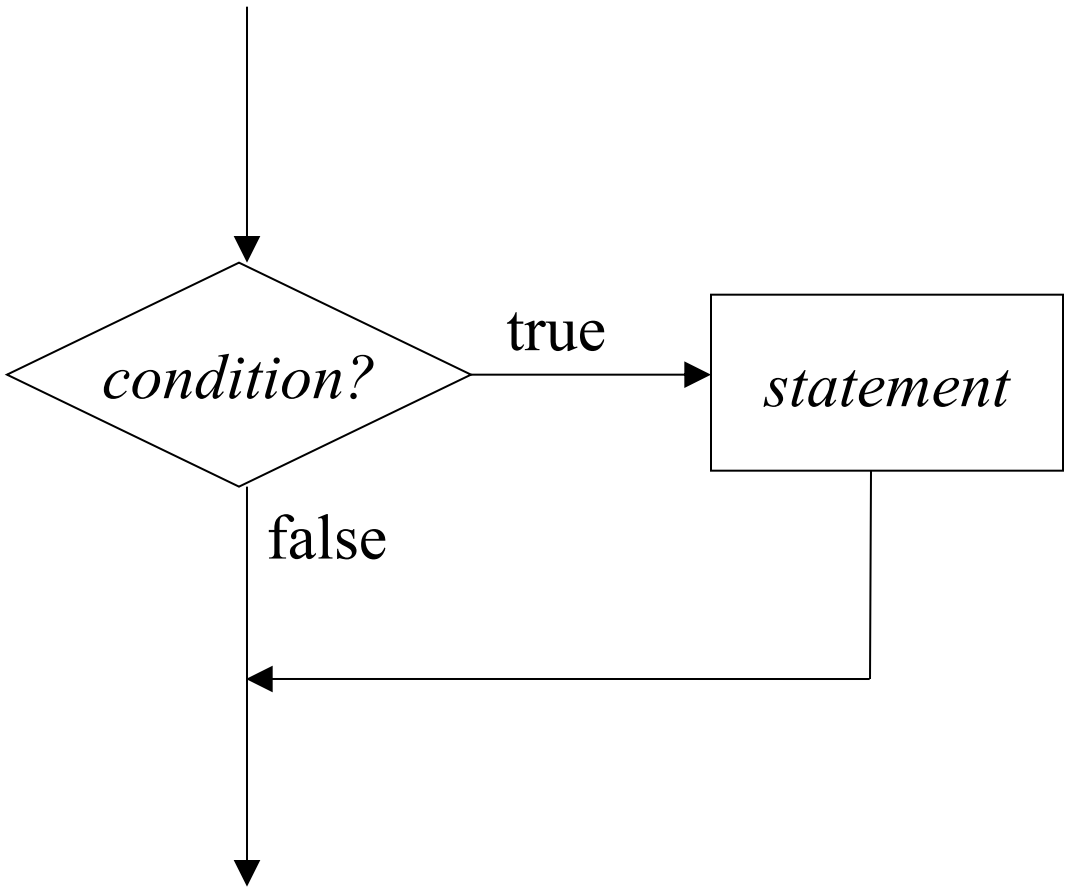
- It's good style to use braces *even if* the if statement controls only a single statement:

```
if (cost > amountInPocket) {
    System.out.println("You can't afford it!");
}
```

- I personally make an exception to this style rule when the controlled statement fits easily on the same line with the if:

```
if (x < 0) x = -x;  // use absolute value of x
```

Flowchart for the **if** statement



The **if-else** statement

- The **if-else** statement chooses which of *two* statements to execute
- The **if-else** statement has the form:

  if (*condition*) *statement-to-execute-if-true* ;
  else *statement-to-execute-if-false* ;

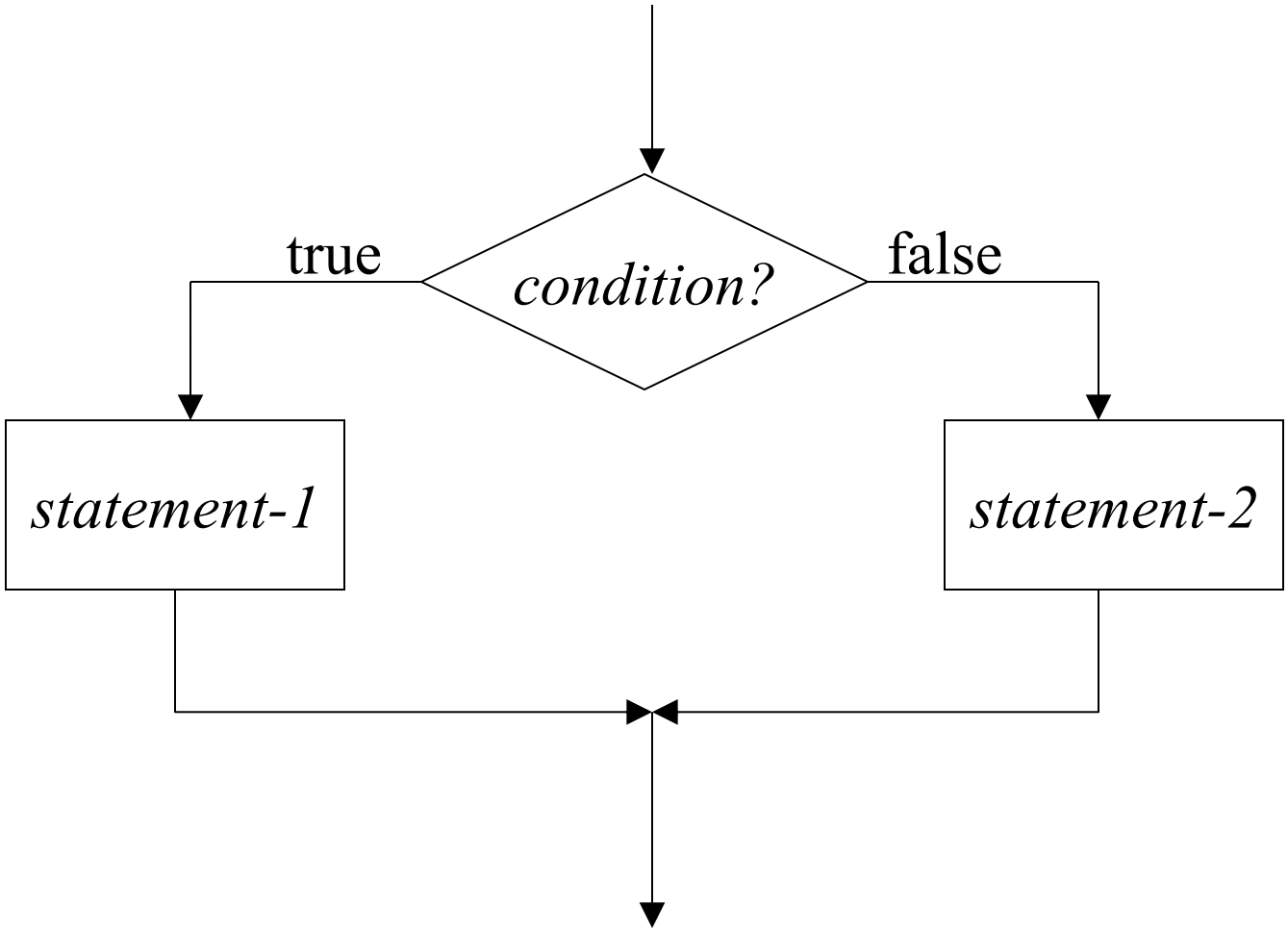
- Either statement (or both) may be a compound statement
- Notice the semicolon after *each* statement

Example **if-else** statements

- ```
  if (x >= 0) absX = x;
  else absX = -x;
  ```

- ```
  if (itemCost <= bankBalance) {
      writeCheck(itemCost);
      bankBalance = bankBalance - itemCost;
  }
  else {
      callHome();
      askForMoreMoney(2 * itemCost);
  }
  ```

Flowchart for the if-else statement



Aside: the “mod” operator

- The modulo, or “mod,” operator returns the remainder of an integer division
- The symbol for this operation is %
- Examples:

  57 % 10 gives 7

  20 % 6 gives 2
- Useful rule: x is divisible by y if x % y == 0

Nesting **if** (or **if-then**) statements

- A year is a leap year if it is divisible by 4 but not by 100, unless it is also divisible by 400

```
if (year % 4 == 0) {
    if (year % 100 == 0) {
        if (year % 400 == 0) leapYear = true;
        else leapYear = false;
    }
    else leapYear = true;
}
else leapYear = false;
```

Operations on **boolean**s

- Assume p and q are booleans
- There are four basic operations on booleans:
  - Negation ("not"):
    !p is true if p is false (and false otherwise)
  - Conjunction ("and"):
    p && q is true if *both* p and q are true
  - Disjunction ("or"):
    p || q is true if *either* of p and q is true
  - Exclusive or ("xor"):
    p ^ q is true if *just one* of p and q is true

Simpler tests

- A simpler leap-year test:

```
if (year % 4 == 0 &&
    (year % 100 != 0 || year % 400 == 0))
        leapYear = true;
else leapYear = false;
```

- An even simpler leap-year test:

```
leapYear = year % 4 == 0 &&
    (year % 100 != 0 || year % 400 == 0);
```

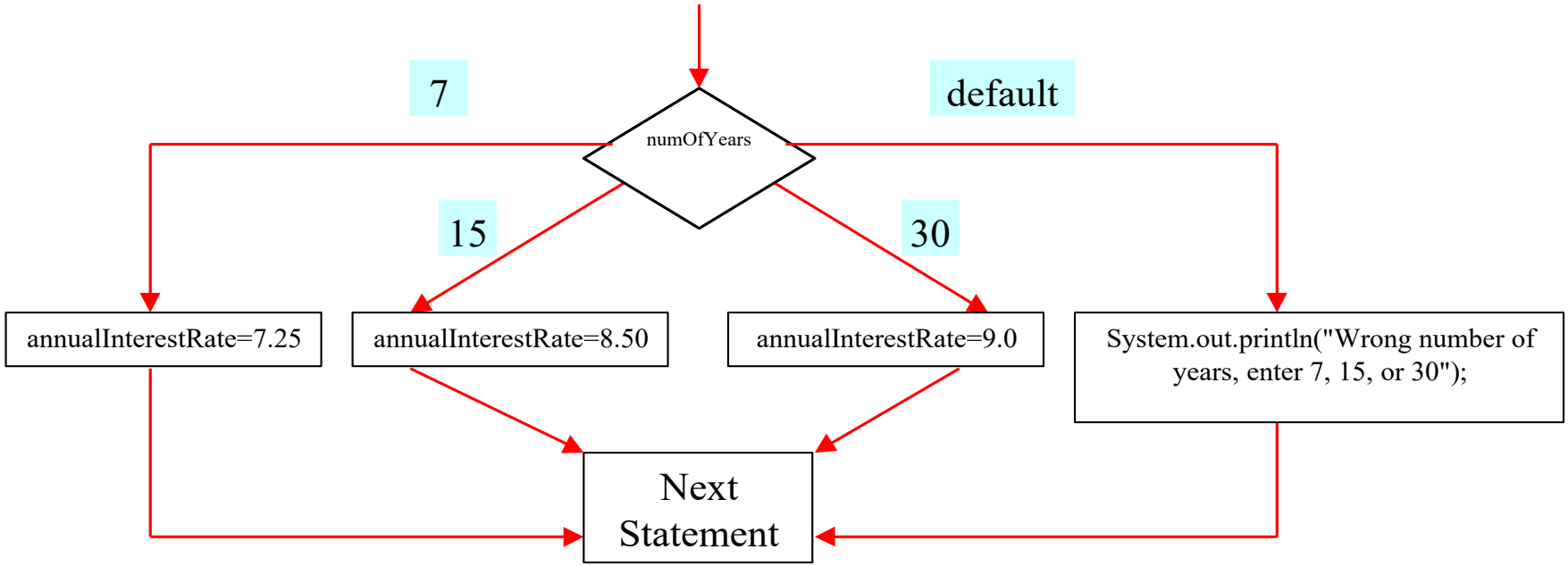
Conditional Operator

```
if (x > 0) y = 1
else y = -1;

is equivalent to

y = (x > 0) ? 1 : -1;
```

switch Statements

```
switch (year)
{
  case 7:  annualInterestRate = 7.25;
           break;
  case 15: annualInterestRate = 8.50;
           break;
  case 30: annualInterestRate = 9.0;
           break;
  default: System.out.println(
   "Wrong number of years, enter 7, 15, or 30");
}
```

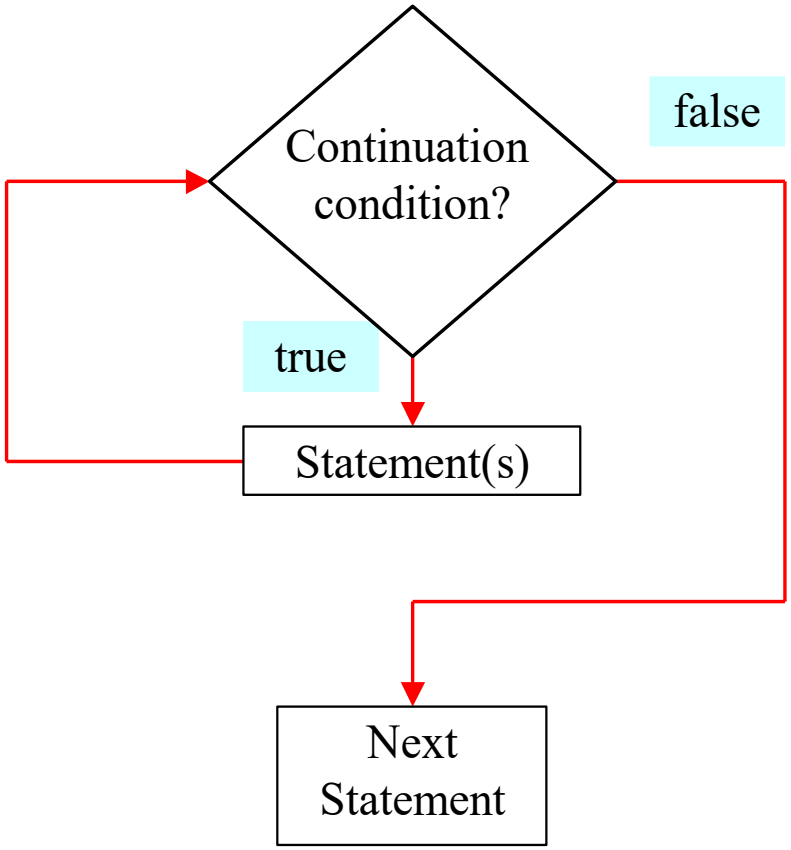
`switch` Statement Flow Chart



Repetitions

- `while` Loops
- `do` Loops
- `for` Loops
- `break` and `continue`

`while` Loop Flow Chart



`while` Loops

```
while (continue-condition)
{
 // loop-body;
}
```

Example 3.2: Using while Loops

TestWhile.java

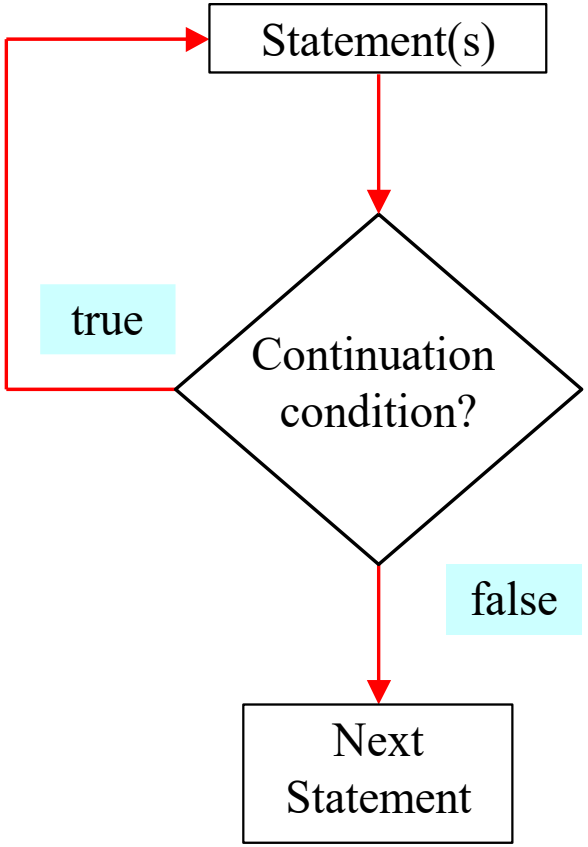
TestWhile

Run

do Loops

```
do
{
   // Loop body;
} while (continue-condition)
```

do Loop Flow Chart



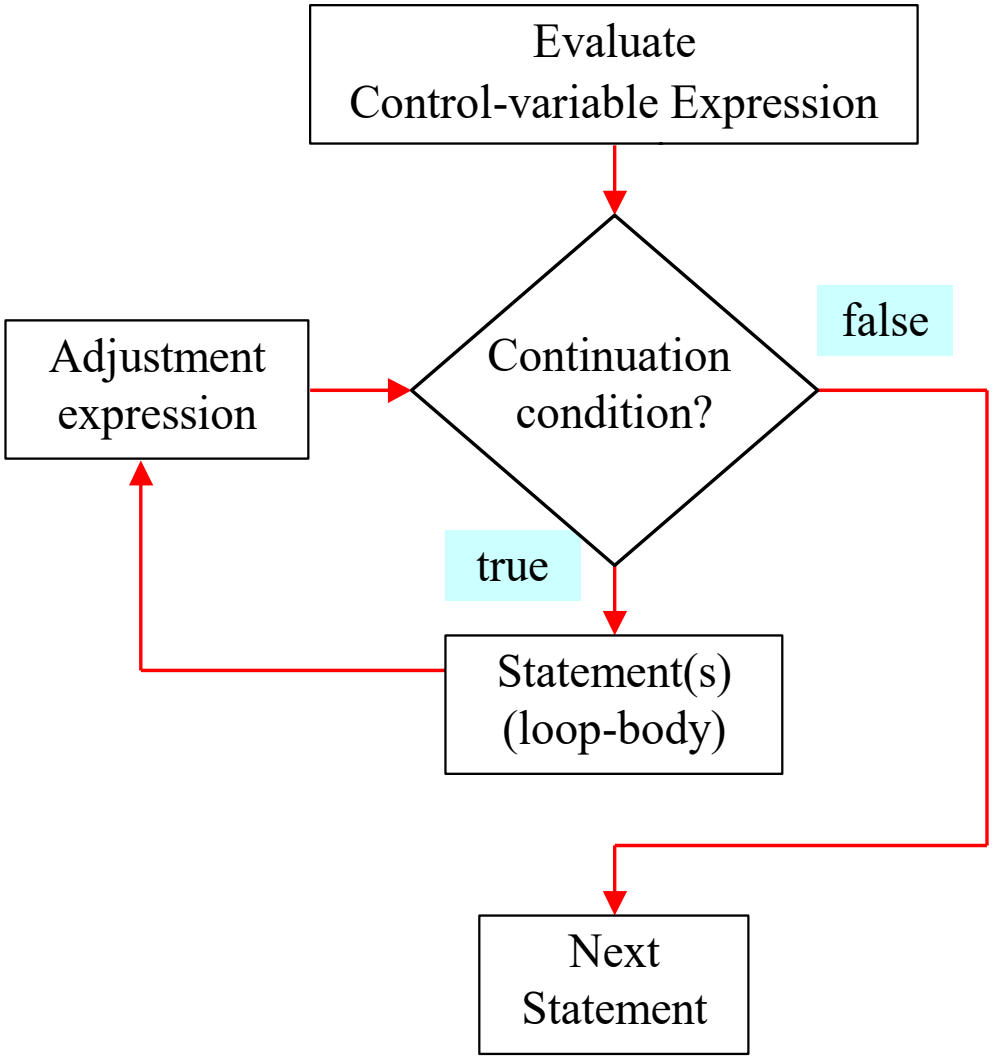
for Loops

```
for (control-variable-initializer;
      continue-condition; adjustment-statement)
{
   //loop body;
}

int i = 0;
while (i < 100)
{
  System.out.println("Welcome to Java! ” + i);
  i++;
}
```

Example:

```
int i;
for (i = 0; i<100; i++)
{
  System.out.println("Welcome to Java! ” + i);
}
```

`for` Loop Flow Chart



`for` Loop Examples

Examples for using the `for` loop:

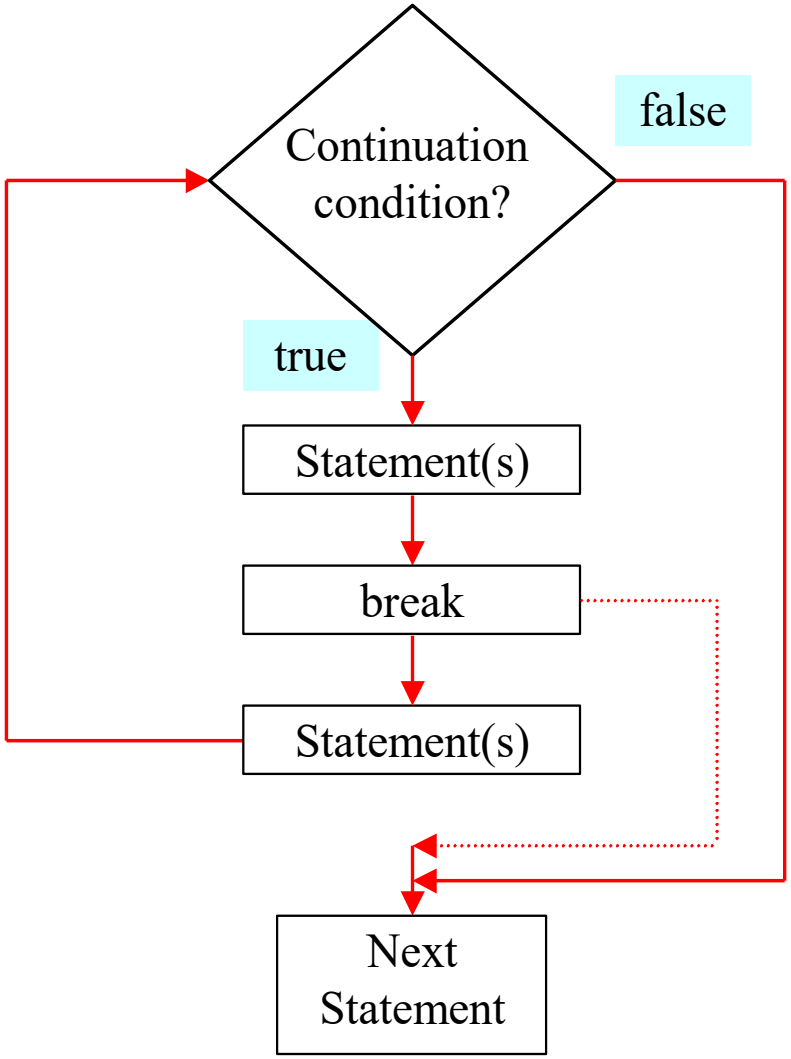
- Example 3.3: Using for Loops

TestSum | Run

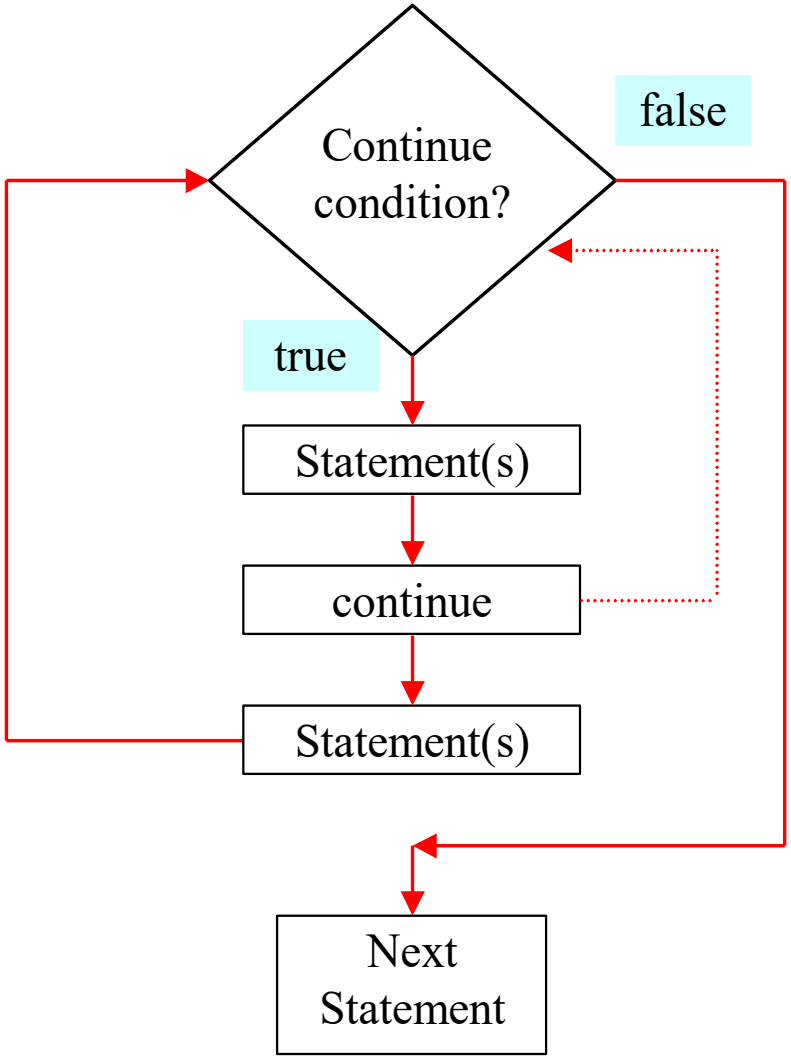
- Example 3.4: Using Nested for Loops

TestMulTable | Run

The `break` Keyword



The `continue` Keyword



Using `break` and `continue`

Examples for using the `break` and `continue` keywords:

- Example 3.5: TestBreak.java

TestBreak

Run

- Example 3.6: TestContinue.java

TestContinue

Run

Access Modifiers

- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

1.Private: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

2.Default: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

3.Protected: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

4.Public: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifiers

- In Java, access modifiers are used to set the accessibility (visibility) of classes, interfaces, variables, methods, constructors, data members, and the setter methods. For example

```
class Animal {
        public void method1() {...}
        private void method2() {...} }
```

Modifier	Description
Default	declarations are visible only within the package (package private)
Private	declarations are visible within the class only
Protected	declarations are visible within the package or all subclasses
Public	declarations are visible everywhere

A **constructor** will have a **access**-control of type default when no **access-modifier** is defined explicitly. So this **constructor** will have a Package Level **Access**.

Access Modifiers

When variables and methods are declared **private**, they cannot be accessed outside of the class. For example,

```
class Data {
   // private variable
   private String name;
}

public class Main {
   public static void main(String[] main){

      // create an object of Data
      Data d = new Data();

      // access private variable and field from another class
      d.name = "Programiz";
   }
}
```

Access Modifiers

When methods and data members are declared **protected,** we can access them within the same package as well as from subclasses. For example,

```
class Animal {
   // protected method
   protected void display() {
      System.out.println("I am an animal");
   }
}

class Dog extends Animal {
   public static void main(String[] args) {

      // create an object of Dog class
      Dog dog = new Dog();
       // access protected method
      dog.display();
   }
}
```

Access Modifiers

When methods, variables, classes, and so on are declared public, then we can access them from anywhere. The public access modifier has no scope restriction. For example,

```
// Animal.java file
// public class
public class Animal {
    // public variable
    public int legCount;

    // public method
    public void display() {
        System.out.println("I am an animal.");
        System.out.println("I have " + legCount + " legs.");
    }
}

// Main.java
public class Main {
    public static void main( String[] args ) {
        // accessing the public class
        Animal animal = new Animal();

        // accessing the public variable
        animal.legCount = 4;
        // accessing the public method
        animal.display();
    }
}
```

Inheritance

When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

When one task is performed by different ways

i.e. known as polymorphism.

For example: to convince the customer

differently, to draw something e.g. shape or rectangle etc.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing.

Encapsulation

Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Arrays

An array is a group of variables that share the same data type, and are referred to by a common name. Arrays of any type can be created and may have one or more dimensions.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

Unlike C/C++, we can get the length of the array using the length member. In C/C++, we need to use the sizeof operator.

A specific element in an array is accessed by its index. The array index ranges from 0 to n−1; therefore, in an array of size 10, the first element is stored at index 0 and the last or the 10th element at index 9.

The following program, Printarr, creates an array of integers, puts some values in it, and prints each value to standard output.

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars.length);
```

Example

```
class Printarr {
  public static void main(String[] args) {
    // declares an array of integers
    int[ ] A;
    // allocates memory for 5 integers
    A = new int[5];
    // initialize elements
    A[0] = 15;//first element
    A[1] = 20;//second element
    A[2] = 25;//third element
    A[3] = 30;//fourth element
    A[4] = 50;//fifth element
System.out.println("Element at index 0: "+ A[0]);
    System.out.println("Element at index 1: "+ A[1]);
    System.out.println("Element at index 2: "+ A[2]);
    System.out.println("Element at index 3: "+ A[3]);
    System.out.println("Element at index 4: "+ A[4]);
  } }
```

The output from this program is:

```
Element at index 0: 15
Element at index 1: 20
Element at index 2: 25
Element at index 3: 30
Element at index 4: 50
```

Copying Arrays

The data from one array can be copied into another array by using the arraycopy method of the System class:

public static void arraycopy(Object src, int srcPos, Object dest, int destPos, int length)

The two Object arguments specify the array to copy from, and the array to copy to. The three int arguments specify the starting position in the source array, the starting position in the destination array, and the number of array elements to copy.

The following program, Copyarr, declares an array of char elements, spelling the word “array”. It uses arraycopy to copy three elements of the first array into the second array:

```
class Copyarr {
  public static void main(String[] args) {
    char[] source = { 'a', 'r', 'r', 'a', 'y' };
    char[] target = new char[3];
    System.arraycopy(source, 0, target, 0, 3);
    System.out.println(new String(target));
  }
}
```

The output from this program is:

arr

Apply the concepts of Packages in any problem

Introduction

- A **java package** is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form, built-in package and user-defined package.
- There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- Here, we will have the detailed learning of creating and using user-defined packages.
- Import class or whole package:
  - `import package.name.Class; // Import a single class import`
  - `package.name.*; // Import the whole package`

Introduction

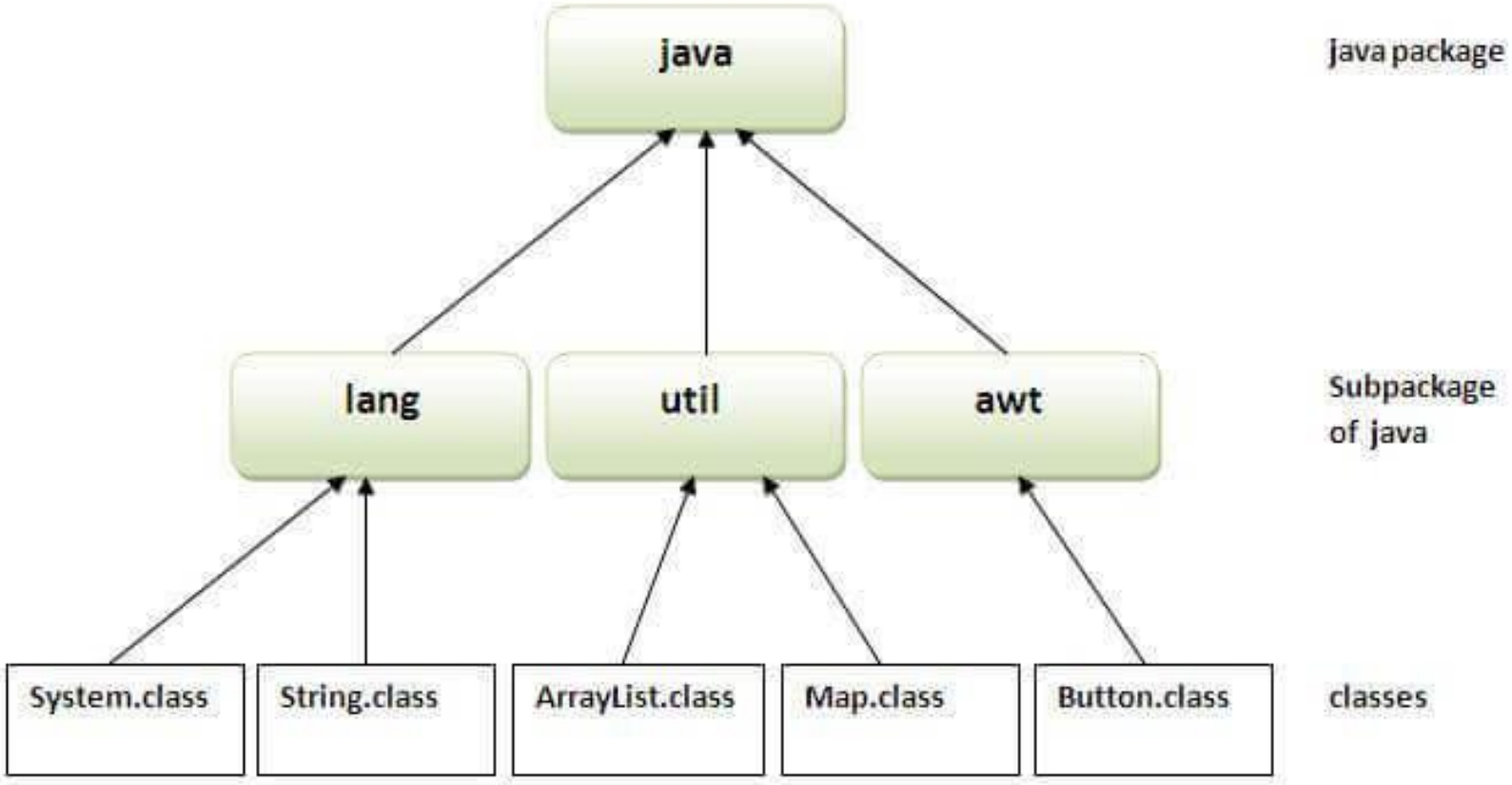
Advantage of Java Package

1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.

2) Java package provides access protection.

3) Java package removes naming collision.

Introduction



Creation Of Jar Files

- In Java source files, the package the file belongs to is specified with the package keyword .
- package java.awt.event;
- JAR Files are created with the jar command-line utility.
- The command "jar cf myPackage.jar *.class"

  compresses all *.class files into the JAR file *myPackage.jar*.

•Some package are

- **lang Package:** Lang stands for language. This package contains those class which are essential for every java program.

  E.g. String and System class

  Lang is the default package of java which means it is optional to import Lang package in program

  Syntax of Import package:-

  import [package name].*; // (all class)
  import [package name].[class name];//(Single class)

•Some package are

- **io package:** io stands for input/output. This package contains those class which are essential for input and output. Some classes of input/output package are;

  E.g. – DataInputStream, BufferReader, DataInoutStramReader

- **util package:** util stands for utility package this package contain different type of classes related to various task.

  E.g.: Date, time, calendar, array, vector. Array List etc.

•Some package are

- **awt package (swing):** awt stands for abstract windowing tool. With the help of this package we can create GUI interface. This package provide many GUI tools.

  E.g.: Command button, choice, list, radio, etc.
  awt contain a sub package name “even”.

- **applet:** with the help of applet function we can create a java program which can be embedded into HTML page by which web browser execute the java program. Web browser must be java enabled.

•Some package are

- **Sql Package:** sql stands for structure query language. This package provides a complete range of classes which is needed for jdbc (java database connectivity). Some important classes of sql package are:

  E.g.: connection, ResultSet, PrepareStatement, etc.

- **Net package:** net stands for networking. This package contains classes to implement or create connection between two or more systems or if we want to implement client server approach then. We can use the classes of net package.

  E.g.: TCP/IP, Request, etc.

How to create package in Java

1. First create a directory within name of package.
2. Create a java file in newly created directory.
3. In this java file you must specify the package name with the help of package keyword.
4. Save this file with same name of public class
   Note: only one class in a program can declare as public.
5. Now you can use this package in your program.

Example Program

- **ALGORITHM**:
- STEP 1:Start the process
- STEP 2:Create a package1, package2 and package 3 to define the triangle shape.
- STEP 3:Create a main class.
- STEP 4: Import the all three packages into main class.
- STEP 5:To get the number of rows as an input. Based on the input the triangle is processed.
- STEP 6: Run the program.
- STEP 7:Stop the process

PACKAGE ONE:

```
package one;
public class cls1
{
int j,k;
public void first(int row,int i)
{
        for(j=1;j<=row-i;j++)
        {
                System.out.print("  ");
        }
        for(k=row-i;k<row;k++)
        {
                System.out.print("* ");
        }
        space(row);
        sec(row,i);
}

void space(int row)
{
 for(j=1;j<=row;j++)
 {
  System.out.print(" ");
 }
}
void sec(int row,int i)
{
        for(k=row-i;k<row;k++)
        {
            System.out.print("* ");
        }
        for(j=1;j<=row-i;j++)
        {
                System.out.print("  ");
        }

}
}
```

PACKAGE TWO:

```
package one.two;
public class cls2
{
int j,k;
public void second(int row,int i)
{
        for(k=row-i;k<row;k++)
        {
                System.out.print("  ");
        }
        for(j=1;j<=row-i;j++)
        {
                System.out.print(" *");
        }
        space(row);
        sec(row,i);
}
void space(int row)
{
 for(j=1;j<=row;j++)
 {
  System.out.print(" ");
 }
}
void sec(int row,int i)
{
        for(j=1;j<=row-i;j++)
        {
                System.out.print("* ");
        }
        for(k=row-i;k<row;k++)
        {
            System.out.print("  ");
        }
}
}
```

PACKAGE THREE:

```
package one.two.three;
public class cls3
{
int j,k;
public void third(int row,int i)
{
        for(j=1;j<=row-i;j++)
        {
                System.out.print(" ");
        }
        for(k=row-i;k<row;k++)
        {
                System.out.print("* ");
        }
        space(row);
        sh(row,i);
}
void space(int row)
{
int j;
 for(j=1;j<=row;j++)
 {
  System.out.print(" ");
 }
}
void sh(int row,int i)
{
        for(k=row-i;k<row;k++)      {
                System.out.print(" ");        }
        for(j=1;j<=row-i;j++)       {
                System.out.print(" *  ");     }
} }
```

MAIN CLASS:

```
import java.io.*;
import one.*;
import one.two.*;
import one.two.three.*;
class m
{
public static void main(String args[])throws IOException
{
int row,i;
BufferedReader sd=new BufferedReader(new
InputStreamReader(System.in));
System.out.println("ENTER number of rows ");
row=Integer.parseInt(sd.readLine());
cls1 a=new cls1();
cls2 b=new cls2();
cls3 c=new cls3();
for(i=0;i<=row;i++)
{
a.first(row,i);
System.out.print("\n");
}
System.out.println("\n");
for(i=0;i<=row;i++)
{
b.second(row,i);
 System.out.print("\n");
}
System.out.println("\n");
for(i=0;i<=row;i++)
{
c.third(row,i);
System.out.print("\n");
}
}
}
```

OUTPUT:

D:\>javac m.java
D:\>java m

ENTER number of rows
4

```
     *   *
    * *   * *
  * * *   * * *
* * * *   * * * *

 * * * *   * * * *
   * * *   * * *
     * *   * *
       *   *

      *  *  *  *
  *     *  *  *
 * *     *  *
 * * *     *
* * * *
```

Apply the concepts of wrapper classes in any problem

Wrapper class in Java

- **Wrapper class in java** provides the mechanism *to convert primitive into object and object into primitive*.
- The main **difference** is that whereas variables can be declared in **Java** as double, short, int, or char, etc., data types, the eight **primitive wrapper classes** create instantiated objects and methods that inherit but hide the eight **primitive** data types, not variables that are assigned data type values.

Primitive Type	Wrapper Class	Primitive Type	Wrapper Class
boolean	Boolean	float	Float
byte	Byte	int	Integer
char	Character	long	Long
double	Double	short	Short

Primitive to Wrapper

```
public class WrapperExample1{
public static void main(String args[]){
//Converting int into Integer
int a=20;
Integer i=Integer.valueOf(a);//converting int into Integer

Integer j=a;//autoboxing, now compiler will write Integer
.valueOf(a) internally

System.out.println(a+" "+i+" "+j);
}}
```

Output:
20 20 20

Wrapper to Primitive

```
public class WrapperExample2{
public static void main(String args[]){
//Converting Integer to int
Integer a=new Integer(3);
int i=a.intValue();//converting Integer to int
int j=a;//unboxing, now compiler will write a.intValue() i
nternally

System.out.println(a+" "+i+" "+j);
}}
```

Output:

3 3 3

Apply the concepts of Interfaces in any application

What is an Interface?

- An interface is similar to an abstract class with the following exceptions:
  - All methods defined in an interface are abstract. Interfaces can contain no implementation
  - Interfaces cannot contain instance variables. However, they can contain public static final variables (ie. constant class variables)
- Interfaces are declared using the "interface" keyword
  - If an interface is public, it must be contained in a file which has the same name.
- Interfaces are more abstract than abstract classes
- Interfaces are implemented by classes using the "implements" keyword.

Declaring an Interface

In Steerable.java:

```
public interface Steerable
{
  public void turnLeft(int degrees);
  public void turnRight(int degrees);
}
```

When a class "implements" an interface, the compiler ensures that it provides an implementation for all methods defined within the interface.

In Car.java:

```
 public class Car extends Vehicle implements Steerable
 {
   public int turnLeft(int degrees)
   {
         [...]
   }

   public int turnRight(int degrees)
   {
         [...]
   }
```

Implementing Interfaces

- A Class can only inherit from one superclass. However, a class may implement several Interfaces
  - The interfaces that a class implements are separated by commas
- Any class which implements an interface must provide an implementation for all methods defined within the interface.
  - NOTE: if an abstract class implements an interface, it NEED NOT implement all methods defined in the interface. HOWEVER, each concrete subclass MUST implement the methods defined in the interface.
- Interfaces can inherit method signatures from other interfaces.

Declaring an Interface

In Car.java:

```
public class Car extends Vehicle implements Steerable, Driveable
{
  public int turnLeft(int degrees)
  {
        [...]
  }

  public int turnRight(int degrees)
  {
        [...]
  }

  // implement methods defined within the Driveable interface
```

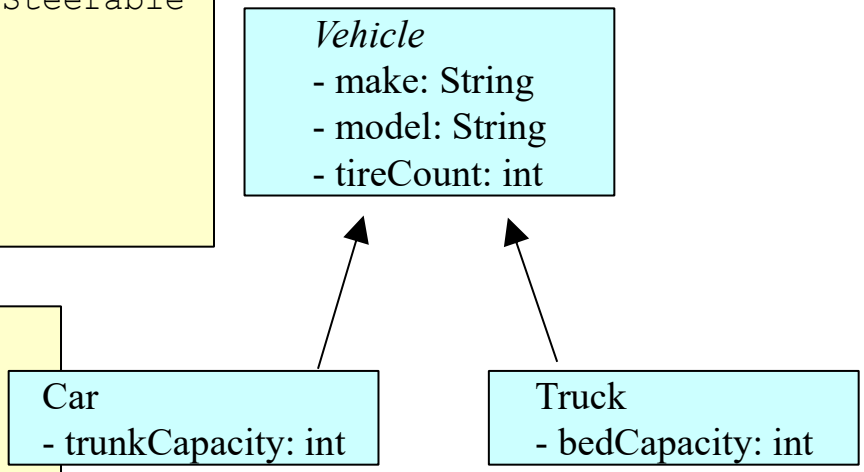
Inheriting Interfaces

- If a superclass implements an interface, it's subclasses also implement the interface

```
public abstract class Vehicle implements Steerable
{
  private String make;
  [...]
```

```
public class Car extends Vehicle
{
  private int trunkCapacity;
  [...]
```

```
public class Truck extends Vehicle
{
  private int bedCapacity;
  [...]
```



Multiple Inheritance?

- Some people (and textbooks) have said that allowing classes to implement multiple interfaces is the same thing as multiple inheritance
- This is NOT true. When you implement an interface:
  - The implementing class does not inherit instance variables
  - The implementing class does not inherit methods (none are defined)
  - The Implementing class does not inherit associations
- Implementation of interfaces is not inheritance. An interface defines a list of methods which must be implemented.

Interfaces as Types

- When a class is defined, the compiler views the class as a new type.
- The same thing is true of interfaces. The compiler regards an interface as a type.
  - It can be used to declare variables or method parameters

```
int i;
Car myFleet[];
Steerable anotherFleet[];

[...]

  myFleet[i].start();

  anotherFleet[i].turnLeft(100);
  anotherFleet[i+1].turnRight(45);
```

Abstract Classes Versus Interfaces

- When should one use an Abstract class instead of an interface?
  - If the subclass-superclass relationship is genuinely an "is a" relationship.
  - If the abstract class can provide an implementation at the appropriate level of abstraction
- When should one use an interface in place of an Abstract Class?
  - When the methods defined represent a small portion of a class
  - When the subclass needs to inherit from another class
  - When you cannot reasonably implement any of the methods

```java
interface MyInterface
{
   public void method1();
   public void method2();
}
class XYZ implements MyInterface
{
  public void method1()
  {
      System.out.println("implementation of method1");
  }
  public void method2()
  {
      System.out.println("implementation of method2");
  }
  public static void main(String arg[])
  {
      MyInterface obj = new XYZ();
      obj. method1();
  }
}
```

```java
public interface Inf1{
   public void method1();
}
public interface Inf2 extends Inf1 {
   public void method2();
}
public class Demo implements Inf2{
  public void method1(){
    //Implementation of method1
  }
  public void method2(){
    //Implementation of method2
  }
}
```

```java
interface A
{
   public void aaa();
}
interface B
{
   public void aaa();
}
class Central implements A,B
{
   public void aaa()
   {
        //Any Code here
   }
   public static void main(String args[])
   {
        //Statements
    }
}
```

```java
interface A
{
   public void aaa();
}
interface B
{
   public int aaa();
}

class Central implements A,B
{

   public void aaa() // error
   {
   }
   public int aaa() // error
   {
   }
   public static void main(String args[])
   {

   }
}
```

Example 1: Below is an example of a Shape interface

```
interface Shape {

        public double area();
        public double volume();
}
```

Below is a Point class that implements the Shape interface.

```
public class Point implements Shape {

        static int x, y;
        public Point() {
                x = 0;
                y = 0;
        }
        public double area() {
                return 0;
        }
        public double volume() {
                return 0;
        }
        public static void print() {
                System.out.println("point: " + x + "," + y);
        }
        public static void main(String args[]) {
                Point p = new Point();
                p.print();
        }
}
```

Describe the concepts of this and super keyword

this keyword

```
class Vehicle{
  Vehicle(){System.out.println("
Vehicle is created");}
}

class Bike6 extends Vehicle{
  int speed;
  Bike6(int speed){
    this.speed=speed;
    System.out.println(speed);
  }
  public static void main(String
 args[]){
   Bike6 b=new Bike6(10);
 }
}
```

super keyword

```
class Vehicle{
  Vehicle(){System.out.println("Vehicle is created");}
}

class Bike5 extends Vehicle{
  Bike5(){
   super();//will invoke parent class constructor
   System.out.println("Bike is created");
  }
  public static void main(String args[]){
   Bike5 b=new Bike5();

}
}
```

```java
class Parentclass {
//Overridden method
void display(){
System.out.println("Parent class method");
}
}

class Subclass extends Parentclass {
//Overriding method
void display(){
System.out.println("Child class method");
}
void printMsg(){
//This would call Overriding method display();
//This would call Overridden method
super.display();
}
public static void main(String args[]){
Subclass obj= new Subclass();
obj.printMsg();
} }
```

Apply the concepts of string handling functions in various applications

String Handling

- Java string is a sequence of characters. They are objects of type String.
- Once a String object is created it cannot be changed. Stings are Immutable.
- To get changeable strings use the class called StringBuffer.
- String and StringBuffer classes are declared final, so there cannot be subclasses of these classes.
- The default constructor creates an empty string.

```
String s = new String();
```

Creating Strings

- `String str = "abc";` is equivalent to:

```
char data[] = {'a', 'b', 'c'};
String str = new String(data);
```

- If data array in the above example is modified after the string object str is created, then str remains unchanged.
- Construct a string object by passing another string object.

  String str2 = new String(str);

String Operations

- The length() method returns the length of the string.

  Eg: System.out.println("Hello".length()); // prints 5
- The + operator is used to concatenate two or more strings.

  Eg: String myname = "Harry"

  String str = "My name is" + myname+ ".";
- For string concatenation the Java compiler converts an operand to a String whenever the other operand of the + is a String object.

String Operations

- Characters in a string can be extracted in a number of ways.

public char **charAt**(int index)

- Returns the character at the specified index. An index ranges from 0 to length() - 1. The first character of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

```
char ch;
ch = “abc”.charAt(1); // ch = “b”
```

String Operations

- **getChars() -** Copies characters from this string into the destination character array.

```
public void getChars(int srcBegin, int srcEnd,
char[] dst, int dstBegin)
```

  - srcBegin - index of the first character in the string to copy.
  - srcEnd - index after the last character in the string to copy.
  - dst - the destination array.
  - dstBegin - the start offset in the destination array.

String Operations

- **equals() -** Compares the invoking string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as the invoking object.

```
public boolean equals(Object anObject)
```

- **equalsIgnoreCase()-** Compares this String to another String, ignoring case considerations. Two strings are considered equal ignoring case if they are of the same length, and corresponding characters in the two strings are equal ignoring case.

```
public boolean equalsIgnoreCase(String
anotherString)
```

String Operations

- **startsWith()** – Tests if this string starts with the specified prefix.

```
public boolean startsWith(String prefix)
"Figure".startsWith("Fig"); // true
```

- **endsWith() -** Tests if this string ends with the specified suffix.

```
public boolean endsWith(String suffix)
"Figure".endsWith("re"); // true
```

String Operations

- **startsWith()** -Tests if this string starts with the specified prefix beginning at a specified index.

```
public boolean startsWith(String prefix,
int toffset)
    prefix - the prefix.
    toffset - where to begin looking in the
    string.

"figure".startsWith("gure", 2); // true
```

String Operations

- **compareTo()** - Compares two strings lexicographically.
  - The result is a negative integer if this String object lexicographically precedes the argument string.
  - The result is a positive integer if this String object lexicographically follows the argument string.
  - The result is zero if the strings are equal.
  - compareTo returns 0 exactly when the equals(Object) method would return true.

```
public int compareTo(String anotherString)
public int compareToIgnoreCase(String str)
```

String Operations

indexOf – Searches for the first occurrence of a character or substring. Returns -1 if the character does not occur.

`public int` **`indexOf`**`(int ch)` – Returns the index within this string of the first occurrence of the specified character.

`public int` **`indexOf`**`(String str)` - Returns the index within this string of the first occurrence of the specified substring.

```
String str = "How was your day today?";
str.indexof('t');
str("was");
```

String Operations

`public int` **`indexOf`**`(int ch, int fromIndex)` – Returns the index within this string of the first occurrence of the specified character, starting the search at the specified index.

`public int` **`indexOf`**`(String str, int fromIndex)` - Returns the index within this string of the first occurrence of the specified substring, starting at the specified index.

String str = “How was your day today?”;
str.indexof(‘a’, 6);
str(“was”, 2);

String Operations

lastIndexOf() –Searches for the last occurrence of a character or substring. The methods are similar to indexOf().

substring() - Returns a new string that is a substring of this string. The substring begins with the character at the specified index and extends to the end of this string.

```
public String substring(int beginIndex)
```

Eg: `"unhappy".substring(2) returns "happy"`

String Operations

- `public String` **`substring`**`(int beginIndex, int endIndex)`

Eg: `"smiles".substring(1, 5)` returns `"mile"`

String Operations

concat() - Concatenates the specified string to the end of this string.

If the length of the argument string is 0, then this String object is returned.

Otherwise, a new String object is created, containing the invoking string with the contents of the str appended to it.

```
public String concat(String str)
"to".concat("get").concat("her") returns
 "together"
```

String Operations

- replace()- Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

```
public String replace(char oldChar, char newChar)
```

```
"mesquite in your cellar".replace('e', 'o')
 returns "mosquito in your collar"
```

String Operations

- **trim() -** Returns a copy of the string, with leading and trailing whitespace omitted.

```
public String trim()

String s = "  Hi Mom!  ".trim();
S = "Hi Mom!"
```

- **valueOf() –** Returns the string representation of the char array argument.

```
public static String valueOf(char[] data)
```

String Operations

- The contents of the character array are copied; subsequent modification of the character array does not affect the newly created string.

Other forms are:

```
public static String valueOf(char c)
public static String valueOf(boolean b)
public static String valueOf(int i)
public static String valueOf(long l)
public static String valueOf(float f)
public static String valueOf(double d)
```

String Operations

- **toLowerCase():** Converts all of the characters in a String to lower case.
- **toUpperCase():** Converts all of the characters in this String to upper case.

```
public String toLowerCase()
public String toUpperCase()

Eg: "HELLO THERE".toLowerCase();
    "hello there".toUpperCase();
```

StringBuffer

- A StringBuffer is like a String, but can be modified.
- The length and content of the StringBuffer sequence can be changed through certain method calls.
- StringBuffer defines three constructors:
  - StringBuffer()
  - StringBuffer(int size)
  - StringBuffer(String str)

StringBuffer Operations

- The principal operations on a StringBuffer are the append and insert methods, which are overloaded so as to accept data of any type.

Here are few append methods:

```
StringBuffer append(String str)
StringBuffer append(int num)
```

- The append method always adds these characters at the end of the buffer.

StringBuffer Operations

- The insert method adds the characters at a specified point.

Here are few insert methods:

```
StringBuffer insert(int index, String str)
StringBuffer append(int index, char ch)
```

Index specifies at which point the string will be inserted into the invoking StringBuffer object.

StringBuffer Operations

- **delete() -** Removes the characters in a substring of this StringBuffer. The substring begins at the specified start and extends to the character at index end - 1 or to the end of the StringBuffer if no such character exists. If start is equal to end, no changes are made.

```
public StringBuffer delete(int start, int end)
```

StringBuffer Operations

- **replace() -** Replaces the characters in a substring of this StringBuffer with characters in the specified String.

```
public StringBuffer replace(int start, int end,
 String str)
```

- **substring() -** Returns a new String that contains a subsequence of characters currently contained in this StringBuffer. The substring begins at the specified index and extends to the end of the StringBuffer.

```
public String substring(int start)
```

StringBuffer Operations

- **reverse() -** The character sequence contained in this string buffer is replaced by the reverse of the sequence.

```
public StringBuffer reverse()
```

- **length() -** Returns the length of this string buffer.

```
public int length()
```

StringBuffer Operations

- **capacity() -** Returns the current capacity of the String buffer. The capacity is the amount of storage available for newly inserted characters.

```
public int capacity()
```

- **charAt() -** The specified character of the sequence currently represented by the string buffer, as indicated by the index argument, is returned.

```
public char charAt(int index)
```

StringBuffer Operations

- **getChars() -** Characters are copied from this string buffer into the destination character array dst. The first character to be copied is at index srcBegin; the last character to be copied is at index srcEnd-1.

```
public void getChars(int srcBegin, int srcEnd,
char[] dst, int dstBegin)
```

- **setLength**() - Sets the length of the StringBuffer.

```
public void setLength(int newLength)
```

Examples: StringBuffer

```
StringBuffer sb = new StringBuffer("Hello");
 sb.length(); // 5
 sb.capacity(); // 21 (16 characters room is
                added if no size is specified)
 sb.charAt(1); // e
 sb.setCharAt(1,'i'); // Hillo
 sb.setLength(2); // Hi
 sb.append("l").append("l"); // Hill
 sb.insert(0, "Big "); // Big Hill
```

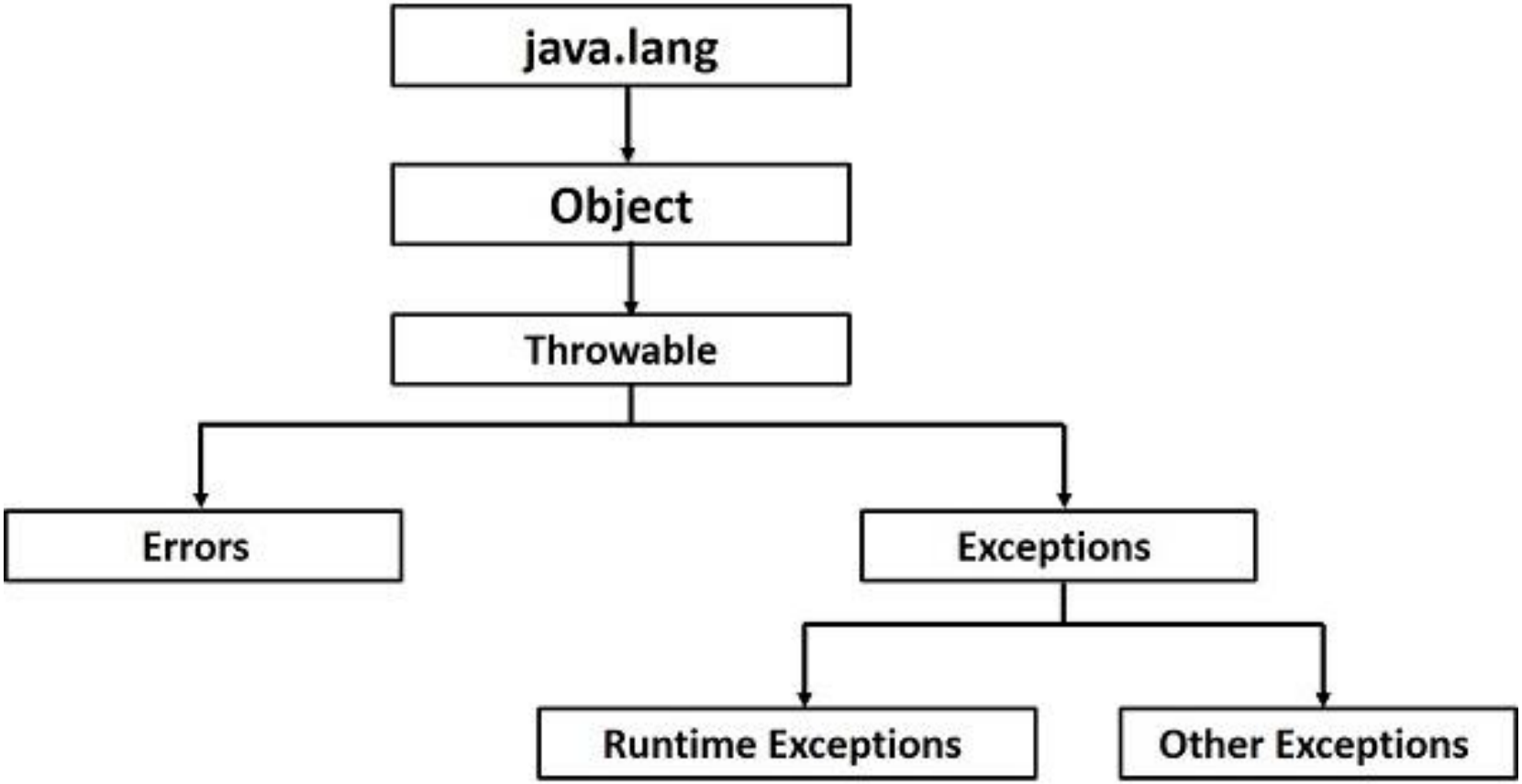
Examples: StringBuffer

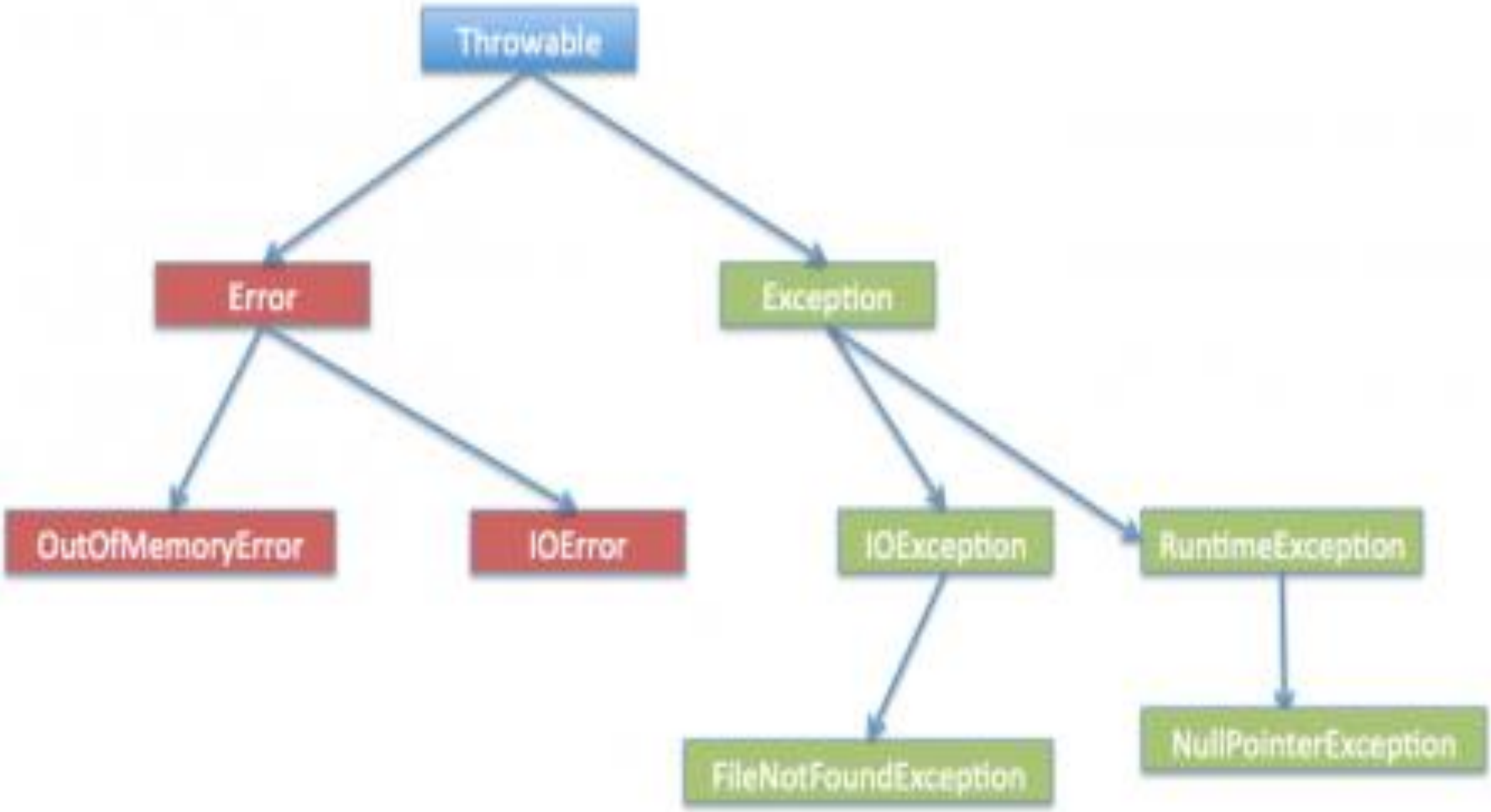
```
sb.replace(3, 11, “”); // Big
sb.reverse(); // gib
```

Exception Handling

What is an exception?

- An *exception* is an error condition that changes the normal flow of control in a program
- Exceptions in Java separates error handling from main business logic
- Based on ideas developed in Ada, Eiffel and C++
- Java has a uniform approach for handling all synchronous errors
  - From very unusual (e.g. out of memory)
  - To more common ones your program should check itself (e.g. index out of bounds)
  - From Java run-time system errors (e.g., divide by zero)
  - To errors that programmers detect and raise deliberately


java.lang
Object
Throwable
Errors
Exceptions
Runtime Exceptions
Other Exceptions


Throwable
Error
Exception
OutOfMemoryError
IOError
IOException
RuntimeException
FileNotFoundException
NullPointerException

scenarios where an exception occurs

- An exception can occur for many different reasons. Following are some scenarios where an exception occurs.
  - A user has entered an invalid data.
  - A file that needs to be opened cannot be found.
  - A network connection has been lost in the middle of communications or the JVM has run out of memory.
- Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner.

Exception Handling Keywords

- **throw** – keyword is used to throw exception to the runtime to handle it.
- **throws** We can provide multiple exceptions in the throws clause and it can be used with main() method also.
- **try-catch** – We use try-catch block for exception handling in our code. try is the start of the block and catch is at the end of try block to handle the exceptions. We can have multiple catch blocks with a try and try-catch block can be nested also. catch block requires a parameter that should be of type Exception.
- **finally** – finally block is optional and can be used only with try-catch block. Since exception halts the process of execution, we might have some resources open that will not get closed, so we can use finally block. finally block gets executed always, whether exception occurred or not.

Types of Exception

- **Checked exceptions** – A checked exception is an exception that occurs at the compile time. These exceptions cannot simply be ignored at the time of compilation; the programmer should take care of (handle) these exceptions.
- **Unchecked exceptions** – An unchecked exception is an exception that occurs at the time of execution. These are also called as **Runtime Exceptions**. These include programming bugs, such as logic errors or improper use of an API. Runtime exceptions are ignored at the time of compilation.

Following is the list of Java Checked Exceptions Defined in java.lang.

Sr.No.	Exception & Description
1	**ClassNotFoundException -** Class not found.
2	**CloneNotSupportedException** Attempt to clone an object that does not implement the Cloneable interface.
3	**IllegalAccessException -** Access to a class is denied.
4	**InstantiationException -** Attempt to create an object of an abstract class or interface.
5	**InterruptedException -** One thread has been interrupted by another thread.
6	**NoSuchFieldException -** A requested field does not exist.
7	**NoSuchMethodException -** A requested method does not exist.

Following is the list of Java Unchecked RuntimeException.

Sr.No.	Exception & Description
1	**ArithmeticException -** Arithmetic error, such as divide-by-zero.
2	**ArrayIndexOutOfBoundsException -** Array index is out-of-bounds.
3	**ArrayStoreException -** Assignment to an array element of an incompatible type.
4	**ClassCastException -** Invalid cast.
5	**IllegalArgumentException -** Illegal argument used to invoke a method.
6	**IllegalMonitorStateException -** Illegal monitor operation, such as waiting on an unlocked thread.
7	**IllegalStateException -** Environment or application is in incorrect state.
8	**IllegalThreadStateException -** Requested operation not compatible with the current thread state.
9	**IndexOutOfBoundsException -** Some type of index is out-of-bounds.
10	**NegativeArraySizeException -** Array created with a negative size.
11	**NullPointerException -** Invalid use of a null reference.
12	**NumberFormatException -** Invalid conversion of a string to a numeric format.
13	**SecurityException -** Attempt to violate security.
14	**StringIndexOutOfBounds -** Attempt to index outside the bounds of a string.
15	**UnsupportedOperationException -** An unsupported operation was encountered.

Catching an exception

```
try {  // statement that could throw an exception
    }
catch (<exception type> e) {
 // statements that handle the exception
}
catch (<exception type> e) { //e higher in hierarchy
 // statements that handle the exception
}
finally {
 // release resources
}
//other statements
```

- At most one catch block executes
- `finally` block always executes once, whether there's an error or not

Execution of try catch blocks

- For normal execution:
  - try block executes, then finally block executes, then other statements execute
- When an error is caught and the catch block throws an exception or returns:
  - try block is interrupted
  - catch block executes (until throw or return statement)
  - finally block executes
- When error is caught and catch block doesn’t throw an exception or return:
  - try block is interrupted
  - catch block executes
  - finally block executes
  - other statements execute
- When an error occurs that is not caught:
  - try block is interrupted
  - finally block executes

Example:

```
try   { p.a = 10;   }
catch (NullPointerException e)
  { System.out.println("p was null"); }
catch (Exception e)
  { System.out.println("other error occurred"); }
catch (Object obj)
  { System.out.println("Who threw that object?"); }
finally { System.out.println(“final processing"); }
System.out.println(“Continue with more statements");
```

Example of Checked exceptions

```
import java.io.File;
import java.io.FileReader;
public class FilenotFound_Demo {
  public static void main(String args[]) {
    File file = new File("E://file.txt");
    FileReader fr = new FileReader(file);
  }
}
```

Output

```
C:\>javac FilenotFound_Demo.java
FilenotFound_Demo.java:8: error: unreported exception
FileNotFoundException; must be caught or declared to be thrown
    FileReader fr = new FileReader(file);
                ^
1 error
```

Example of unchecked exceptions

```
public class Unchecked_Demo {

  public static void main(String args[]) {
    int num[] = {1, 2, 3, 4};
    System.out.println(num[5]);
  }
}
```

Output

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: 5
        at Exceptions.Unchecked_Demo.main(Unchecked_Demo.java:8)
```

DivideByZero exception

```
public class MainClass {
  public static void main(String args[]) {

    int urAns, urDiv;

    try {

      urDiv = 0;
      urAns = 25 / urDiv;

      System.out.println("Do you really think this will print out? No! It won't!");

    }

    catch (ArithmeticException e) {

      System.out.println("Division by zero not Possible!");

    }

    System.out.println("This will print out after Exception Handling");
  }

}
```

NullPointerException

```
package beginnersbook.com;
class Exception2
{
public static void main(String args[])
{
        try{
                String str=null;
                System.out.println (str.length());
        }catch(NullPointerException e){
                System.out.println("NullPointerException..");
        }
}
}
```

Output:

NullPointerException..

ArithmeticException

```
class ExceptionDemo1
{
  public static void main(String args[])
  {
    try{
      int num1=30, num2=0;
      int output=num1/num2;
      System.out.println ("Result = " +output);
    }
    catch(ArithmeticException e){
      System.out.println ("Arithmetic Exception: You can't divide an integer by 0");
    }
  }
}
```

Output of above program:

Arithmetic Exception: You can't divide an integer by 0

ArrayIndexOutOfBounds Exception

```
class ExceptionDemo2
{
  public static void main(String args[])
  {
    try{
      int a[]=new int[10];
      //Array has only 10 elements
      a[11] = 9;
    }
    catch(ArrayIndexOutOfBoundsException e){
       System.out.println ("ArrayIndexOutOfBounds");
    }
  }
}
```

Output:

ArrayIndexOutOfBounds

NumberFormat Exception

```
class ExceptionDemo3
{
  public static void main(String args[])
  {
    try{
          int num=Integer.parseInt ("XYZ") ;
          System.out.println(num);
    }catch(NumberFormatException e){
           System.out.println("Number format exception occurred");
     }
  }
}
```

Output:

Number format exception occurred

StringIndexOutOfBound Exception

```
class ExceptionDemo4
{
  public static void main(String args[])
  {
    try{
            String str="easysteps2buildwebsite";
            System.out.println(str.length());;
            char c = str.charAt(0);
            c = str.charAt(40);
            System.out.println(c);
    }catch(StringIndexOutOfBoundsException e){
             System.out.println("StringIndexOutOfBoundsException!!");
     }
  }
}
```

Output:

22

StringIndexOutOfBoundsException!!

Apply the throw keyword with given problem.

Throwing and catching

- An error can *throw* an *exception*

```
throw <exception object>;
```

- By default, exceptions result in the thread terminating after printing an error message
- However, exception handlers can *catch* specified exceptions and recover from error

```
catch (<exception type> e) {
 //statements that handle the
exception
}
```

Throwing an exception

- Example creates a subclass of Exception and throws an exception:

```
class MyException extends Exception {   }

class MyClass   {
    void oops()
    {  if (/* no error occurred */)
       { /* normal processing */ }
               else { /* error occurred */
          throw new MyException();
       }
     }  //oops
 }//class MyClass
```

Apply the concepts of Custom Exception in given scenario.

- If you are creating your own Exception that is known as custom exception or user-defined exception. Java custom exceptions are used to customize the exception according to user need.
- By the help of custom exception, you can have your own exception and message.

```
class InvalidAgeException extends Exception{
 InvalidAgeException(String s){
  super(s);
 }
}

class TestCustomException1{
    static void validate(int age)throws InvalidAgeException{
    if(age<18)
     throw new InvalidAgeException("not valid");
    else
     System.out.println("welcome to vote");
   }
    public static void main(String args[]){
     try{
     validate(13);
     }catch(Exception m){System.out.println("Exception occured: "+m);}
      System.out.println("rest of the code...");
  }
}
```

Output:
Exception occured: InvalidAgeException:not valid
rest of the code...

Multithreading

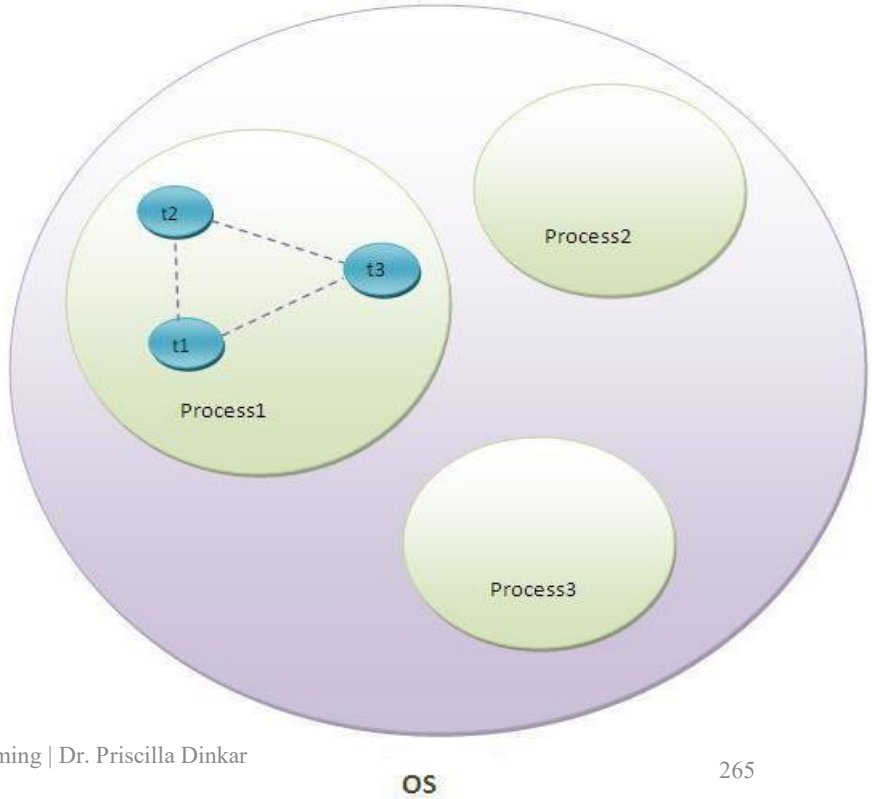
Explain the concepts of Life Cycle of a Thread.

- **Multithreading in java** is a process of executing multiple threads simultaneously.
- Thread is basically a lightweight sub-process, a smallest unit of processing.
- Multiprocessing and multithreading, both are used to achieve multitasking.
- But we use multithreading than multiprocessing because threads share a common memory area.
- They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.
- Java Multithreading is mostly used in games, animation etc.

What is Thread in java

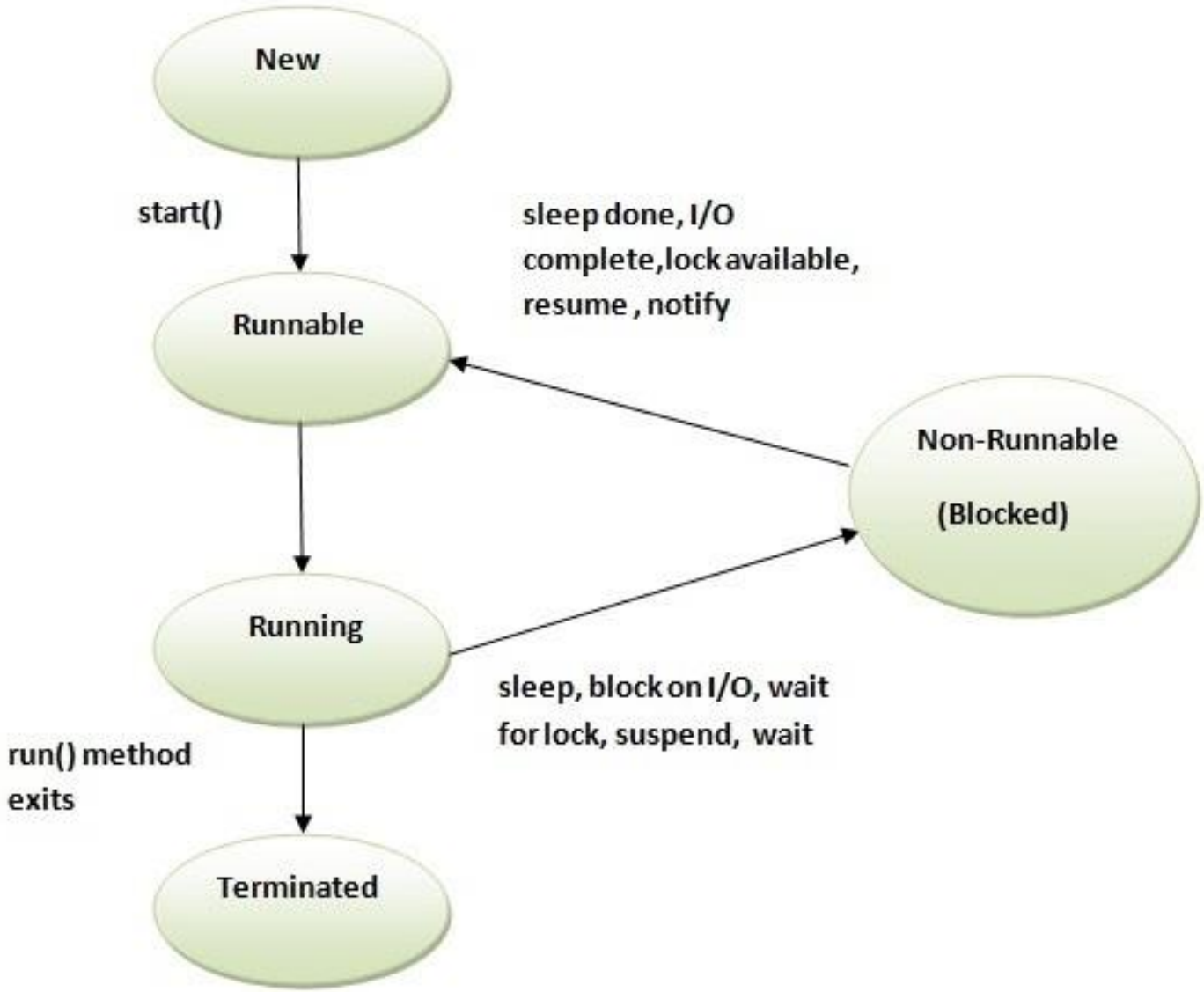
- A thread is a lightweight sub process, a smallest unit of processing. It is a separate path of execution.
- Threads are independent, if there occurs exception in one thread, it doesn't affect other threads. It shares a common memory area.

- As shown in the above figure, thread is executed inside the process.
- There is context-switching between the threads.
- There can be multiple processes inside the OS and one process can have multiple threads.



Life cycle of a Thread (Thread States)

- A thread can be in one of the five states. According to sun, there is only 4 states in **thread life cycle in java** new, runnable, non-runnable and terminated. There is no running state.
- But for better understanding the threads, we are explaining it in the 5 states.
- The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:
    1. New
    2. Runnable
    3. Running
    4. Non-Runnable (Blocked)
    5. Terminated


New
start()
Runnable
sleep done, I/O complete, lock available, resume , notify
Non-Runnable
(Blocked)
Running
sleep, block on I/O, wait for lock, suspend, wait
run() method exits
Terminated

Life cycle of a Thread (Thread States)

1) New
   - The thread is in new state if you create an instance of
   - Thread class but before the invocation of start() method.
2) Runnable
   - The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.
3) Running
   - The thread is in running state if the thread scheduler has selected it.
4) Non-Runnable (Blocked)
   - This is the state when the thread is still alive, but is currently not eligible to run.
5) Terminated
   - A thread is in terminated or dead state when its run() method exits.

Apply the concepts of Creating Thread and Thread Schedular in given situation.

How to create thread

There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface.

Thread class:

1. Thread class provide constructors and methods to create and perform operations on a thread.
2. Thread class extends Object class and implements Runnable interface.

Commonly used Constructors of Thread class:

- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r,String name)

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread.JVM calls the run() method on the thread.
3. **public void sleep(long miliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. **public void join():** waits for a thread to die.
5. **public void join(long miliseconds):** waits for a thread to die for the specified miliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.
10. **public Thread currentThread():** returns the reference of currently executing thread.
11. **public int getId():** returns the id of the thread.
12. **public Thread.State getState():** returns the state of the thread.
13. **public boolean isAlive():** tests if the thread is alive.
14. **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. **public void suspend():** is used to suspend the thread(depricated).
16. **public void resume():** is used to resume the suspended thread(depricated).
17. **public void stop():** is used to stop the thread(depricated).
18. **public boolean isDaemon():** tests if the thread is a daemon thread.
19. **public void setDaemon(boolean b):** marks the thread as daemon or user thread.
20. **public void interrupt():** interrupts the thread.
21. **public boolean isInterrupted():** tests if the thread has been interrupted.
22. **public static boolean interrupted():** tests if the current thread has been interrupted.

Runnable interface:

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named run().

1. **public void run():** is used to perform action for a thread.

Starting a thread:

start() method of Thread class is used to start a newly created thread. It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.

Sr. No.	Key	Thread	Runnable
1	Basic	Thread is a class. It is used to create a thread	Runnable is a functional interface which is used to create a thread
2	Methods	It has multiple methods including start() and run()	It has only abstract method run()
3		Each thread creates a unique object and gets associated with it	Multiple threads share the same objects.
4	Memory	More memory required	Less memory required
5	Limitation	Multiple Inheritance is not allowed in java hence after a class extends Thread class, it can not extend any other class	If a class is implementing the runnable interface then your class can extend another class.

1) Java Thread Example by extending Thread class

```
class Multi extends Thread{
public void run(){
System.out.println("thread is running...");
}
public static void main(String args[]){
Multi t1=new Multi();
t1.start();
 }
}
```

Output:thread is running...

2) Java Thread Example by implementing Runnable interface

```
class Multi3 implements Runnable{
public void run(){
System.out.println("thread is running...");
}

public static void main(String args[]){
Multi3 m1=new Multi3();
Thread t1 =new Thread(m1);
t1.start();
 }
}
```

Output:thread is running...

Advantages of Java Multithreading

1. It **doesn't block the user** because threads are independent and you can perform multiple operations at same time.

2. You **can perform many operations together so it saves time**.

3. Threads are **independent** so it doesn't affect other threads if exception occur in a single thread.

Thread Scheduler in Java

- **Thread scheduler** in java is the part of the JVM that decides which thread should run.
- There is no guarantee that which runnable thread will be chosen to run by the thread scheduler.
- Only one thread at a time can run in a single process.
- The thread scheduler mainly uses preemptive or time slicing scheduling to schedule the threads.

Difference between preemptive scheduling and time slicing

- Under preemptive scheduling, the highest priority task executes until it enters the waiting or dead states or a higher priority task comes into existence.
- Under time slicing, a task executes for a predefined slice of time and then reenters the pool of ready tasks. The scheduler then determines which task should execute next, based on priority and other factors.

Thread Scheduler Java

- In **Java** program, you create **threads** but they are not executed by **Java** alone.
- The **scheduler** maintains a pool of **threads**.
- Priority of **thread**.
- The JVM is based on preemptive and priority based **scheduling** algorithm.
- main is a method for us, but main is a **thread** for JVM.

WorkerThread.java

```
package com.journaldev.threads;
import java.util.Date;
public class WorkerThread implements Runnable{
private String command;
public WorkerThread(String s){
        this.command=s;
    }
@Override
    public void run() {
        System.out.println(Thread.currentThread().getName()+" Start. Time = "+new Date());
        processCommand();
        System.out.println(Thread.currentThread().getName()+" End. Time = "+new Date());
    }
    private void processCommand() {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    @Override
    public String toString(){
        return this.command;
    }
}
```

Apply the concepts of Sleeping a thread and Joining a thread in given problem.

Sleep method in java

The sleep() method of Thread class is used to sleep a thread for the specified amount of time.

Syntax of sleep() method in java

The Thread class provides two methods for sleeping a thread:

- public static void sleep(long miliseconds)throws InterruptedException
- public static void sleep(long miliseconds, int nanos)throws InterruptedException

Example of sleep method in java

```
class TestSleepMethod1 extends Thread{
 public void run(){
  for(int i=1;i<5;i++){
    try{Thread.sleep(500);}
catch(InterruptedException e){
System.out.println(e);}
    System.out.println(i);
  }
 }
 public static void main(String args[]){
  TestSleepMethod1 t1=new TestSleepMethod1();
  TestSleepMethod1 t2=new TestSleepMethod1();

  t1.start();
  t2.start();
 }
}
```

Output:
1
1
2
2
3
3
4
4

Can we start a thread twice

No. After starting a thread, it can never be started again. If you does so, an *IllegalThreadStateException* is thrown.
In such case, thread will run once but for second time, it will throw exception.

Let's understand it by the example given below:

```
public class TestThreadTwice1 extends Thread{
 public void run(){
  System.out.println("running...");
 }
 public static void main(String args[]){
  TestThreadTwice1 t1=new TestThreadTwice1();
  t1.start();
  t1.start();
 }
}
```

Test it Now

running Exception in thread "main" java.lang.IllegalThreadStateException

Joining Threads in Java

java.lang.Thread class provides the join() method which allows one thread to wait until another thread completes its execution. If **t** is a Thread object whose thread is currently executing, then **t.join();** it causes the current thread to pause its execution until thread it join completes its execution.

There are three overloaded join functions.

1.join(): It will put the current thread on wait until the thread on which it is called is dead. If thread is interrupted then it will throw InterruptedException.

Syntax: `public final void join()`

2.join(long millis) :It will put the current thread on wait until the thread on which it is called is dead or wait for specified time (milliseconds).

Syntax: `public final synchronized void join(long millis)`

3.join(long millis, int nanos): It will put the current thread on wait until the thread on which it is called is dead or wait for specified time (milliseconds + nanos).

Syntax: `public final synchronized void join(long millis, int nanos)`

```
// Java program to explain the
// concept of joining a thread.
import java.io.*;

// Creating thread by creating the
// objects of that class
class ThreadJoining extends Thread
{
    @Override
    public void run()
    {
        for (int i = 0; i < 2; i++)
        {
            try
            {
                Thread.sleep(500);
                System.out.println("Current Thread:
"+Thread.currentThread().getName());
            }

            catch(Exception ex)
            {
                System.out.println("Exception has" + " been caught" + ex);
            }
            System.out.println(i);
        }
    }
}
```

```
class GFG
{
    public static void main (String[] args)
    {
          // creating two threads
        ThreadJoining t1 = new ThreadJoining();
        ThreadJoining t2 = new ThreadJoining();
        ThreadJoining t3 = new ThreadJoining();
         // thread t1 starts
        t1.start();
          // starts second thread after when
        // first thread t1 is died.
        try
{
            System.out.println("Current Thread: "+ Thread.currentThread().getName());
            t1.join();
        }
          catch(Exception ex)
        {
            System.out.println("Exception has " + "been caught" + ex);
        }
          // t2 starts
        t2.start();

        // starts t3 after when thread t2 is died.
        try
        {
            System.out.println("Current Thread: "+ Thread.currentThread().getName());
            t2.join();
        }
          catch(Exception ex)
        {
            System.out.println("Exception has been" + " caught" + ex);
        }

        t3.start();
    }
}
```

Output:

```
Current Thread: main
Current Thread: Thread-0
0
Current Thread: Thread-0
1
Current Thread: main
Current Thread: Thread-1
0
Current Thread: Thread-1
1
Current Thread: Thread-2
0
Current Thread: Thread-2
1
```

Apply the concepts of Thread Priority in given problem.

Priority of a Thread (Thread Priority):

- Each thread have a priority.
- Priorities are represented by a number between 1 and 10.
- In most cases, thread schedular schedules the threads according to their priority (known as preemptive scheduling).
- But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

- 3 constants defiend in Thread class:

public static int MIN_PRIORITY
public static int NORM_PRIORITY
public static int MAX_PRIORITY

- Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

Example of priority of a Thread:

```
class TestMultiPriority1 extends Thread{
 public void run(){
   System.out.println("running thread name is:"+Thread.currentThread().getName())
;
   System.out.println("running thread priority is:"+Thread.currentThread().getPriorit
y());

  }
 public static void main(String args[]){
  TestMultiPriority1 m1=new TestMultiPriority1();
  TestMultiPriority1 m2=new TestMultiPriority1();
  m1.setPriority(Thread.MIN_PRIORITY);
  m2.setPriority(Thread.MAX_PRIORITY);
  m1.start();
  m2.start();

 }
}
```

Output:

```
running thread name is:Thread-0
running thread priority is:10
running thread name is:Thread-1
running thread priority is:1
```

Inheritance

- Inheritance in Java is the method to create a hierarchy between classes by inheriting from other classes.
- Java Inheritance is transitive - so if Sedan extends Car and Car extends Vehicle, then Sedan is also inherited from the Vehicle class. The Vehicle becomes the superclass of both Car and Sedan.
- Inheritance represents the IS-A relationship which is also known as a parent-child relationship.
- Why use inheritance in java
  - For Method Overriding (so runtime polymorphism can be achieved).
  - For Code Reusability.

Inheritance

- The syntax of Java Inheritance

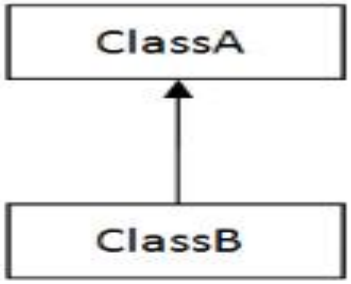
```
class Subclass-name extends Superclass-name
{  //methods and felds  }
```

- The extends keyword indicates that you are making a new class that derives from an existing class.
- The meaning of "extends" is to increase the functionality.

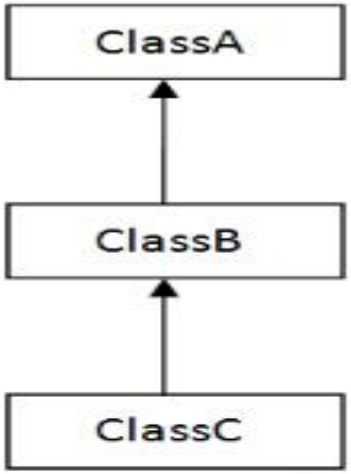
Inheritance

```
class Employee{
        float salary=40000;
}
class Programmer extends Employee{
        int bonus=10000;
        public static void main(String args[]){
                Programmer p=new Programmer();
                System.out.println("Programmer salary
                                                is:"+p.salary);
                System.out.println("Bonus of Programmer
                                                is:"+p.bonus);
  }

}
```

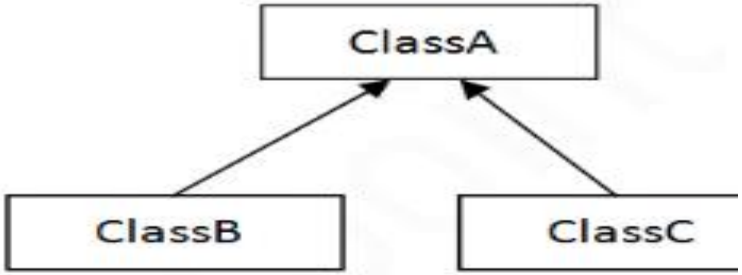
Types of Inheritance



1) Single



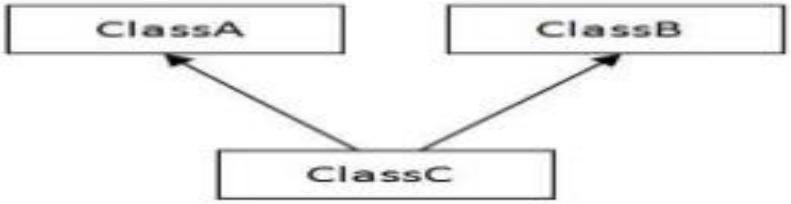
2) Multilevel



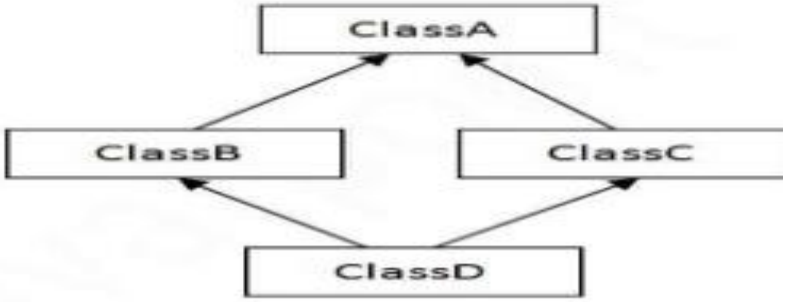
3) Hierarchical

Note: Multiple inheritance is not supported in Java through class.

When one class inherits multiple classes, it is known as multiple inheritance.
For Example:



4) Multiple



5) Hybrid

Types of Inheritance

Single Inheritance Example

When a class inherits another class, it is known as a *single inheritance*. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

File: TestInheritance.java

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output:

```
barking...
eating...
```

Types of Inheritance

Multilevel Inheritance Example

When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

File: TestInheritance2.java

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

```
weeping...
barking...
eating...
```

Types of Inheritance

Q) Why multiple inheritance is not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java.

Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

```
class A{
void msg(){System.out.println("Hello");}
}
class B{
void msg(){System.out.println("Welcome");}
}

class C extends A,B{//suppose if it were

 public static void main(String args[]){
  C obj=new C();
  obj.msg();//Now which msg() method would be invoked?
}
}
```

```
Compile Time Error
```

Types of Inheritance

Hierarchical Inheritance Example

- When two or more classes inherits a single class, it is known as *hierarchical inheritance*.
- In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

Types of Inheritance

Hierarchical Inheritance

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}}
```

Output:

meowing... eating...

Apply the concepts of synchronized method in given problem.

Synchronization in Java

- Synchronization in java is the capability *to control the access of multiple threads to any shared resource*.
- Java Synchronization is better option where we want to allow only one thread to access the shared resource.

Why use Synchronization

The synchronization is mainly used to

1. To prevent thread interference.
2. To prevent consistency problem.

Synchronization in Java

Types of Synchronization

There are two types of synchronization

1. Process Synchronization
2. Thread Synchronization

Here, we will discuss only thread synchronization.

Thread Synchronization

There are two types of thread synchronization mutual exclusive and inter-thread communication.

1. Mutual Exclusive
    1. Synchronized method.
    2. Synchronized block.
    3. static synchronization.
2. Cooperation (Inter-thread communication in java)

Mutual Exclusive

- Mutual Exclusive helps keep threads from interfering with one another while sharing data.
- This can be done by three ways in java:

    1. by synchronized method
    2. by synchronized block
    3. by static synchronization

Concept of Lock in Java

- Synchronization is built around an internal entity known as the lock or monitor.
- Every object has an lock associated with it.
- By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.
- From Java 5 the package java.util.concurrent.locks contains several lock implementations.

Understanding the problem without Synchronization

```
Class Table{

void printTable(int n){//method not synchroni
    zed
  for(int i=1;i<=5;i++){
    System.out.println(n*i);
    try{
     Thread.sleep(400);
    }catch(Exception e){System.out.println(e);}
        }
  }
 }
class MyThread1 extends Thread{
Table t;
MyThread1(Table t){
this.t=t;
}
public void run(){
t.printTable(5);
}
}
```

O/P:

```
5
100
10
200
15
300
20
400
25
500
```

```
class MyThread2 extends Thread{
Table t;
MyThread2(Table t){
this.t=t;
}
public void run(){
t.printTable(100);
}
}

class TestSynchronization1{
public static void main(String args[]){
Table obj = new Table();//only one object
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
}
}
```

Java synchronized method

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

Example of java synchronized method

```
class Table{
 synchronized void printTable(int n){//sync method
   for(int i=1;i<=5;i++){
     System.out.println(n*i);
     try{
      Thread.sleep(400);
     }catch(Exception e){System.out.println(e);}
   }
  }}

class MyThread1 extends Thread{
Table t;
MyThread1(Table t){
this.t=t;
}
public void run(){
t.printTable(5);
}
  }
```

O/P:
5
10
15
20
25
100
200
300
400
500

```
class MyThread2 extends Thread{
Table t;
MyThread2(Table t){
this.t=t;
}
public void run(){
t.printTable(100);
}
}

public class TestSynchronization2{
public static void main(String args[]){
Table obj = new Table();//only one object
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
}
}
```

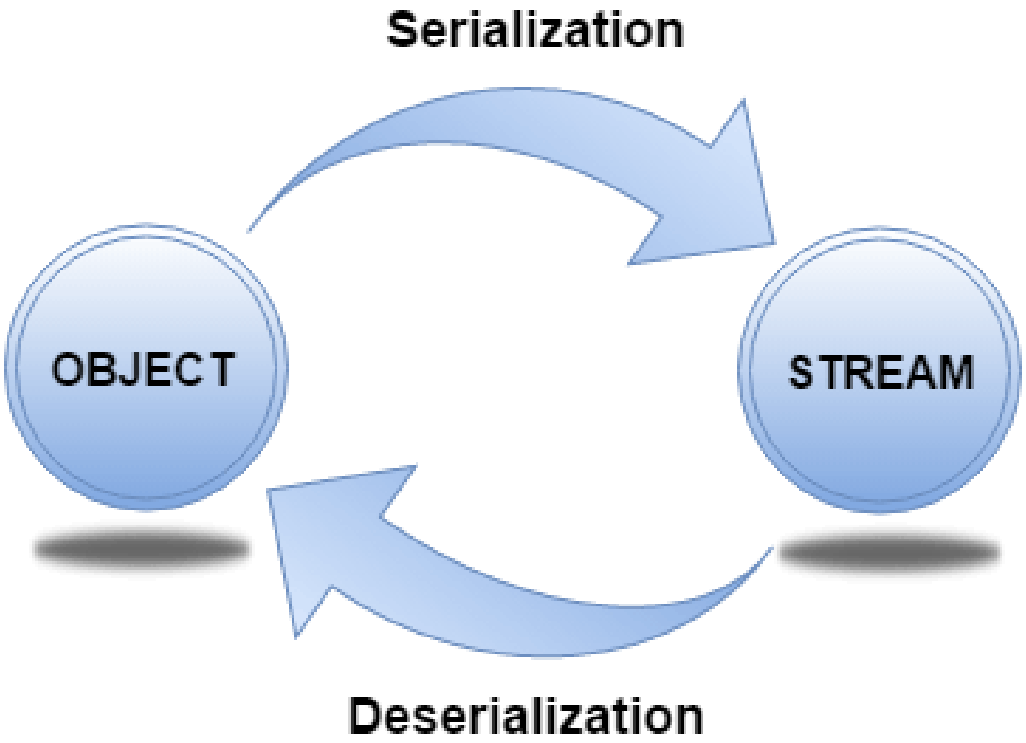
Serialization and Deserialization

Serialization and Deserialization

- **Serialization in Java** is a mechanism of *writing the state of an object into a byte-stream*. It is mainly used in Hibernate, RMI, JPA, EJB and JMS technologies.
- The reverse operation of serialization is called *deserialization* where byte-stream is converted into an object. The serialization and deserialization process is platform-independent, it means you can serialize an object on one platform and deserialize it on a different platform.
- For serializing the object, we call the **writeObject()** method of *ObjectOutputStream* class, and for deserialization we call the **readObject()** method of *ObjectInputStream* class.
- We must have to implement the *Serializable* interface for serializing the object.

Advantages of Java Serialization

- It is mainly used to travel object's state on the network (that is known as marshalling).



java.io.Serializable interface

Serializable is a marker interface (has no data member and method). It is used to "mark" Java classes so that the objects of these classes may get a certain capability.
The **Cloneable** and **Remote** are also marker interfaces.

The **Serializable** interface must be implemented by the class whose object needs to be persisted.

The String class and all the wrapper classes implement the *java.io.Serializable* interface by default.

java.io.Serializable interface

Let's see the example given below:

Student.java

```
import java.io.Serializable;
public class Student implements Serializable{
 int id;
 String name;
 public Student(int id, String name) {
  this.id = id;
  this.name = name;
 }
}
```

In the above example, ***Student*** class implements Serializable interface. Now its objects can be converted into stream. The main class implementation of is showed in the next code.

ObjectOutputStream class

The ObjectOutputStream class is used to write primitive data types, and Java objects to an OutputStream. Only objects that support the java.io.Serializable interface can be written to streams.

Constructor

1) public ObjectOutputStream(OutputStream out) throws IOException {}	It creates an ObjectOutputStream that writes to the specified OutputStream.

Important Methods

Method	Description
1) public final void writeObject(Object obj) throws IOException {}	It writes the specified object to the ObjectOutputStream.
2) public void flush() throws IOException {}	It flushes the current output stream.
3) public void close() throws IOException {}	It closes the current output stream.

ObjectInputStream class

An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

Constructor

1) public ObjectInputStream(InputStream in) throws IOException {}	It creates an ObjectInputStream that reads from the specified InputStream.

Important Methods

Method	Description
1) public final Object readObject() throws IOException, ClassNotFoundException{}	It reads an object from the input stream.
2) public void close() throws IOException {}	It closes ObjectInputStream.

Example of Java Serialization

In this example, we are going to serialize the object of ***Student*** class from above code. The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object. We are saving the state of the object in the file named f.txt.

Persist.java

```
1.import java.io.*;
2.class Persist{
3. public static void main(String args[]){
4.  try{
5.  //Creating the object
6.  Student s1 =new Student(211,"ravi");
7.  //Creating stream and writing the object
8.  FileOutputStream fout=new FileOutputStream("f.txt");
9.  ObjectOutputStream out=new ObjectOutputStream(fout);
10.  out.writeObject(s1);
11.  out.flush();
12.  //closing the stream
13.  out.close();
14.  System.out.println("success");
15.  }catch(Exception e){System.out.println(e);}
16. }
17.}
```

Output:

success

Example of Java Deserialization

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. Let's see an example where we are reading the data from a deserialized object.

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. Let's see an example where we are reading the data from a deserialized object.

Depersist.java

```
import java.io.*;
class Depersist{
 public static void main(String args[]){
  try{
  //Creating stream to read the object
  ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
  Student s=(Student)in.readObject();
  //printing the data of the serialized object
  System.out.println(s.id+" "+s.name);
  //closing the stream
  in.close();
  }catch(Exception e){System.out.println(e);}
 }
}
```

Output:

211 ravi

END